

GPT-6 ASTRA

The Complete Zero-to-Hero Guide

Master AI Before the World Catches Up

13 Complete Chapters | Beginner to Advanced
Master Prompts, Computer Use, Agents & Automation
Slash Cached Input Costs by Up to 90% with Prompt Caching
Build AI-Powered Businesses With New Possibilities
30-Day Mastery Roadmap Included

BY

Faisal Fayaz Lone

PREMIUM EDITION | SEPTEMBER 2026

GPT-6 ASTRA

The Complete Zero-to-Hero Guide

Copyright (c) 2026. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Disclaimer: This book is an independent educational guide. It is not affiliated with, endorsed by, or sponsored by OpenAI.

All product names, logos, and brands are property of their respective owners. GPT-6 Astra, ChatGPT, Codex, and related marks are trademarks of OpenAI. This book references publicly available information from OpenAI's official documentation, system card, and launch announcements current as of September 2026.

The information in this book is provided for educational purposes only. Pricing, features, and capabilities are subject to change. Always verify current details on OpenAI's official website before making business or financial decisions.

First Edition: September 2026

Premium Edition

PRINT THIS BOOK

If you would like to own this ebook as a printed physical book, or if you want to distribute printed copies for your team, classroom, or organization, you can reach out to arrange printing and delivery.

Contact the author to discuss printing options, bulk orders, or custom editions:

Email: lonefaisal977@gmail.com

First Edition: September 2026

Author: Faisal Fayaz Lone

Premium Edition

TABLE OF CONTENTS

Introduction: Why This Book Exists

What You Will Achieve

Chapter 1: The Dawn of Astra

What Is GPT-6 Astra?

The Evolution from GPT-4 to Astra

Why Astra Matters: The Numbers

The AGI Conversation

What Makes Astra Different

Chapter 2: Getting Started -- Access and Setup

Choosing Your Access Path

ChatGPT Plans and Pricing

API Access

Azure and AWS Bedrock

GitHub Copilot

Your First API Call

Chapter 3: Understanding Astra's Architecture

The Million-Token Context Window

Five Levels of Reasoning

Knowledge and Training

Input and Output Capabilities

The Responses API

Built-in Tools

Chapter 4: Mastering Prompts

Why Prompting Changed with Astra

Initiative and Follow-Through

Instruction Priority

Writing Style and Personality

The Prompt Structure Framework

Copy-Paste Prompt Templates

Chapter 5: Slashing Costs -- Astra on a Budget

Understanding the Pricing Model

The 272K Threshold Trap

- Prompt Caching: Your Biggest Lever
- Reasoning Effort Optimization
- Batch and Flex Modes
- Model Routing Strategy
- Real Cost Calculations

Chapter 6: Computer Use and Browser Automation

- What Is Computer Use?
- The Observe-Decide-Execute Loop
- Code Execution vs Structured Tool
- Building a Computer Use Workflow
- Safety Controls

Chapter 7: Building Your First AI Agent

- What Is an AI Agent?
- The Responses API
- Function Calling
- Structured Outputs
- Step-by-Step Agent Build

Chapter 8: Automating Your Business with Astra

- CRM and Sales Automation
- Customer Support
- Research and Reporting
- Document Creation
- Marketing Campaigns
- Financial Analysis

Chapter 9: Business Ideas Built on Astra

- AI Operations Agency
- Automated Content Studio
- One-Person Consulting Firm
- AI-Powered SaaS
- Automated QA Service
- AI Research Service

Chapter 10: Coding and Software Engineering

- Codex with Astra
- Debugging Workflows
- Searchable Context Feature
- PR Creation and Review
- Testing Automation

Chapter 11: Advanced Automation Patterns

- Multi-Agent Orchestration
- The Manager-Agent Pattern

- Long-Running Workflows
- MCP Integration
- Mid-Turn Steering
- Asynchronous Function Calls

Chapter 12: Safety, Ethics and Alignment

- The Critical Classification
- Confirmation Policies
- Enterprise Controls
- Responsible Use Guidelines
- Misalignment Monitoring

Chapter 13: The Future and Your Action Plan

- What Is Coming Next
- How to Stay Ahead
- Your 30-Day Mastery Roadmap
- Final Thoughts

Resources and References

Back Cover

CHAPTER

Introduction

Why This Book Exists

On September 3, 2026, OpenAI released GPT-6 Astra, and the world changed. Not with a dramatic press event or a stadium-size launch, but with a quiet blog post that contained a sentence nobody had ever written before about a shipping AI model: "Welcome to the AGI era."

GPT-6 Astra is not just another chatbot upgrade. It is the first AI model that can operate your computer, browse the web, write and debug code, fill out forms, update your CRM, build websites, create presentations, run quality checks, and carry multi-step professional workflows from start to finish -- all while costing less per completed task than the model that came before it. OpenAI's president Greg Brockman said it might already qualify as artificial general intelligence under the company's own definition: a system that performs economically valuable work as well as or better than humans.

Whether or not you agree with that claim, the practical reality is undeniable. Astra changes what a single person can accomplish. Tasks that used to require a team of developers, researchers, and analysts can now be completed by one person with the right prompts and the right workflow. Businesses that previously needed too much manual labor to be practical for a small team become more viable. And the people who learn to use Astra effectively today will have an advantage over those who wait.

This book exists to give you that advantage. It is designed for the absolute beginner who has never written a single prompt, and it carries you all the way to advanced automation patterns, multi-agent systems, and business ventures built on Astra's capabilities. Every fact, benchmark, pricing detail, and API specification in this book comes from OpenAI's official documentation, the GPT-6 Astra system card, and the launch announcements -- all current as of September 2026.

You do not need a computer science degree. You do not need prior AI experience. You do not need to know how to code, though this book will teach you enough to use Astra's API and build working agents. What you need is curiosity, a willingness to experiment, and the desire to be among the first to master the most powerful AI tool ever created.

By the end of this book, you will go from zero to hero. You will understand what Astra is, how it works, how to use it, how to afford it, and how to build a career or business around it. Let us begin.

Who This Book Is For

This book serves four kinds of readers, and each will find a different path through it:

- **The complete beginner.** You have heard about AI but never used it beyond casual chat. Start at Chapter 1 and read in order. By Chapter 5, you will be writing prompts that professionals use. By Chapter 7, you will be building agents. No technical background is required -- every concept is explained before it is used.
- **The professional.** You already use ChatGPT or another AI tool for work. You want to go deeper: automate workflows, reduce costs, and use Astra's advanced capabilities. Read Chapters 4 and 5 first (prompting and cost), then jump to the chapters that match your function: sales, support, marketing, finance, or engineering.
- **The entrepreneur.** You want to build a business on Astra or add AI capabilities to an existing one. Read Chapters 1, 4, 5, and 9 in that order. Chapter 9 is written for you: eight business models with startup steps, cost estimates, and pricing guidance.
- **The developer.** You build software and want to integrate Astra into applications. Read Chapters 5, 7, 10, and 11 for API details, agent architecture, coding workflows, and advanced patterns. The code examples are in Python and use the official OpenAI SDK.

Whatever your background, the 30-day roadmap in Chapter 13 gives you a concrete day-by-day plan. Do not read this book passively. Open ChatGPT or your API console alongside it and try each example as you read it. The difference between reading about Astra and using it is the difference between watching a sport and playing it.

WHAT YOU WILL ACHIEVE

After completing this book, you will be able to do the following with confidence and skill:

- 1 Understand what GPT-6 Astra is, how it differs from every previous AI model, and why OpenAI calls it a generational leap toward AGI.
- 2 Access Astra through ChatGPT, the OpenAI API, Microsoft Azure, AWS Bedrock, or GitHub Copilot, and choose the right plan for your needs and budget.
- 3 Write professional-grade prompts using OpenAI's official prompting framework, including autonomy scope, instruction priority, completion criteria, and verification rules.
- 4 Reduce cached input-token costs by up to 90 percent using prompt caching, reasoning effort tuning, batch processing, and smart model routing -- without sacrificing output quality.
- 5 Use Astra's computer-use capabilities to automate browser tasks, fill forms, update CRMs, and operate desktop applications that have no API.
- 6 Build functional AI agents using the Responses API, function calling, and structured outputs, and deploy them in production environments.
- 7 Automate core business operations: sales prospecting, customer support, research, reporting, document creation, and marketing campaigns.
- 8 Identify and launch business ventures that Astra makes more practical, including AI operations agencies, automated content studios, and one-person consulting firms. Pricing figures are illustrative examples.
- 9 Use Codex with Astra for software engineering: debugging, testing, pull request creation, and large-scale refactoring with searchable context.
- 10 Design multi-agent systems with the manager-agent pattern, long-running workflows, MCP integration, mid-turn steering, and asynchronous function calls.
- 11 Apply safety best practices: confirmation policies, enterprise controls, responsible use guidelines, and understanding of Astra's Critical cybersecurity classification.
- 12 Execute a 30-day mastery roadmap that takes you from your first prompt to a working AI-powered business or automation system.

How to Read This Book

If you are new to AI, read chapters 1 through 5 in order. They build on each other and give you the foundation you need. If you already have experience with GPT models, you can skip to chapter 4 (prompts) or chapter 5 (cost optimization) and branch out from there. Every chapter is self-contained enough to reference later, but the first five form the critical path.

CHAPTER 1

The Dawn of Astra

What GPT-6 Astra Is and Why It Changes Everything

What Is GPT-6 Astra?

GPT-6 Astra is the most capable artificial intelligence model ever built and released to the public. Developed by OpenAI and launched on September 3, 2026, Astra is the successor to GPT-5.6 Sol and represents what OpenAI describes as a generational leap across computer use, browsing, software engineering, cybersecurity, science, and professional work.

Unlike previous AI models that primarily generated text in response to prompts, Astra can take actions. It can navigate websites, click buttons, fill out forms, update customer records in a CRM, organize calendars, conduct online research, draft documents, build spreadsheets, create presentations, generate websites, run quality assurance checks, install software, and troubleshoot problems it sees on screen. It does this by combining advanced reasoning with the ability to interact with software interfaces -- even when those applications do not have a developer API.

OpenAI has stated that Astra was pretrained at the Stargate facility and described it as their largest training run ever, attributed to OpenAI researcher Aidan Clark in press reporting. OpenAI has not officially published the GPU count in its blog posts or model documentation. Treat the specific GPU figure as press-reported, not as an official OpenAI specification.

In OpenAI's own words from the launch announcement: "GPT-6 Astra brings together years of research and big bets across pre-training, reinforcement learning, and alignment. Astra is state-of-the-art on computer use, browsing, software engineering, cybersecurity, science, and professional work."

The Evolution: From GPT-4 to Astra

To understand why Astra is a leap rather than an increment, it helps to see the trajectory. GPT-4, released in March 2023, was the model that made the world take AI seriously. It could write essays, pass professional exams, and generate code, but it could not take actions. It lived in a text box and responded to text with text.

GPT-5 and then GPT-5.6 Sol brought improvements in reasoning, coding, and tool use. Sol introduced function calling, structured outputs, and basic computer-use capabilities. But each of these models still required significant human guidance for complex multi-step tasks. You had to prompt for each step, verify each result, and manually carry context from one interaction to the next.

Astra changes the equation. OpenAI researcher Aidan Clark described the capability gain from Sol to Astra as larger than the jump from earlier models to Sol. The model can now take an open-ended challenge, break it into steps, evaluate options, create a plan, execute it across multiple applications, verify its own work, and produce a polished result -- all with minimal human intervention.

Why Astra Matters: The Numbers

OpenAI published benchmark results that show Astra's capabilities in concrete terms. These are self-reported numbers from OpenAI's own evaluations, but they are the most specific data available:

Benchmark	What It Tests	GPT-6 Astra	GPT-5.6 Sol
ARC-AGI-3	Abstract reasoning	99.9%	Lower
FrontierMath Tier 4	Mathematics	98%	Lower
ExploitBench	Cybersecurity	100%	Lower
OSWorld 2.0 (offline)	Computer use	72.6%	65.7%
ScreenSpot-Pro	Screen understanding	92.7%	76.9%
Terminal-Bench 4.0	Terminal tasks	57.9%	37.3%
Agents' Last Exam	Professional tasks	59.3%	53.6%
AutomationBench	Workflow automation	41.4%	18.1%
GPQA Diamond	Expert knowledge	96.0%	Lower
BrowseComp	Web browsing	91.5%	90.4%

The numbers tell a clear story. On computer-use tasks, Astra scores 72.6 percent on OSWorld 2.0 compared to Sol's 65.7 percent, and it completes tasks in roughly 40 minutes versus 75 minutes for Sol -- about 47 percent less time per task. On Terminal-Bench 4.0, which tests complex terminal-based tasks including software engineering and system configuration, Astra reaches 57.9 percent compared to Sol's 37.3 percent. On AutomationBench, which measures workflow automation, Astra more than doubles Sol's score: 41.4 percent versus 18.1 percent.

Perhaps most strikingly, Astra saturates several benchmarks. It scores 99.9 percent on ARC-AGI-3, 98 percent on FrontierMath Tier 4, and 100 percent on ExploitBench. These are not marginal improvements -- they are ceiling-level results that mean the benchmarks themselves may need to be redesigned to continue measuring progress.

The AGI Conversation

OpenAI's president Greg Brockman closed the launch briefing with the words: "Welcome to the AGI era." He acknowledged that there is no single clearly defined AGI moment -- the team originally thought there would be an obvious threshold everyone would recognize when OpenAI was founded, but the transition turned out to be more gradual than expected.

OpenAI once defined AGI as "an automated system that can perform all economically valuable work as well as or better than humans." Whether Astra meets that bar is debated. Independent testers note that on broad knowledge tests, Astra is not dramatically smarter than its predecessor. Its advantage is not raw intelligence but agency: the ability to take actions, complete multi-step workflows, and carry tasks to completion with fewer tokens and fewer retries.

The Practical Takeaway

You do not need to decide whether Astra is AGI to benefit from it. What matters is that Astra can do things no previous model could do: operate your computer, complete multi-hour workflows, and deliver finished work that requires less human correction. The people who learn to direct that capability will replace the people who do not, regardless of where the philosophical line of AGI falls.

The AI Landscape: A Brief History

To appreciate what Astra represents, it helps to understand how we got here. In seven years, AI language models went from generating barely coherent paragraphs to operating computers autonomously. Here is the timeline of the milestones that matter:

Year	Milestone	Why It Mattered
2018	Transformer architecture matured	Google's research enabled modern LLMs
2020	GPT-3 (175B parameters)	First model to generate human-like text at scale
2022 Nov	ChatGPT launched	AI became a consumer product; 100M users in 2 months
2023 Mar	GPT-4 released	Passed professional exams; world took AI seriously
2024	Tool use and function calling	Models could take actions, not just generate text
2025	GPT-5 series	Reasoning models; test-time compute scaling
2026 Feb	GPT-5.6 family (Sol, Terra, Luna)	Tiered models for different use cases
2026 Sep	GPT-6 Astra	First model at Critical cyber capability; computer use at production quality

Notice the acceleration. It took four years from GPT-3 to GPT-4, two years from GPT-4 to GPT-5, and months between recent major releases. Each generation compresses the time between breakthroughs. Astra is not the end of this curve -- it is the model that closed the gap between "AI that describes" and

"AI that does." What comes next will build on the same foundations you learn in this book.

The Economics of a New Capability Class

Every previous advance in AI has followed the same economic pattern: a capability that was expensive becomes cheap, and new businesses form around the price drop. When web hosting went from thousands of dollars to pennies, SaaS was born. When app stores gave independent developers distribution, the app economy emerged. When AI could write code, no-code tools multiplied.

Astra represents the same kind of price collapse, but for a different input: skilled labor. Tasks that required a team -- research, analysis, document preparation, quality testing, customer communication -- can now be performed by one person directing Astra. The cost per task drops from \$30 to \$10.67 in OpenAI's own example calculation, and the person's time drops from 45 minutes to 10 minutes.

The businesses in Chapter 9 are the ones that form around this price drop. They were not possible before because the labor cost exceeded what customers would pay. With Astra, the economics flip. Understanding this pattern -- expensive capability becomes cheap, new businesses form -- is more valuable than any single business idea, because it lets you spot the next opportunity yourself.

The Road to Launch

Astra's release did not happen in isolation. Following OpenAI's Hugging Face incident in July 2026, the company delayed the release of their next model to add more safeguards. On September 1, 2026, OpenAI published "Path to Astra," explaining that after additional evaluation, they believed Astra met the Critical cybersecurity capability threshold under their Preparedness Framework -- the first model to receive that designation.

The designation required stronger safeguards during development and before release: stricter isolation, checkpoint encryption, universal monitoring of full trajectories including chains of thought, and a blocking alignment evaluation process before internal use. OpenAI stated that Astra was not involved in the Hugging Face incident, but the event shaped how carefully the company approached this launch.

The rollout itself was staged. Astra was released as a limited preview to select organizations on September 3, with a stable public release to paid users the following day, September 4. Access to its most advanced cybersecurity capabilities remains restricted to a group of alpha testers, with defensive access expanding through the Daybreak Blue program. Enterprise workspace administrators must explicitly enable the model; it is off by default at launch.

Why the Staged Rollout Matters to You

The staged rollout means that Astra is new to everyone -- including the companies you compete with. Early adopters have a window of several weeks to several months where they can build workflows and expertise that others have not started. That window is closing as you read this. The best time to start learning Astra was on September 3. The second best time is today.

What Makes Astra Different

Five things separate Astra from every AI model that came before it:

- **Computer use at production quality.** Astra can navigate websites and desktop applications, click buttons, type text, fill forms, and read the results. It works through the same interfaces humans use, even when no API exists.
- **Long-horizon task completion.** Astra can carry multi-step workflows across hours, maintaining context and making decisions without human guidance at each step. It plans, validates, and self-corrects.
- **Token efficiency.** Despite costing 2.5 times more per token than GPT-5.6 Sol, Astra completes tasks in fewer tokens with fewer retries, resulting in a lower cost per completed task.
- **Searchable context across windows.** In Codex, Astra can keep notes across context windows and search earlier windows for requirements or test results that were not captured in summaries.
- **Most aligned model yet.** Astra respects task boundaries, asks focused questions when the answer could change the outcome, and produces 89 percent fewer unintended outcomes than Sol on adversarial workplace tests.

These five capabilities combine to create something genuinely new: an AI that does not just answer questions but completes work. That distinction is the foundation of everything in this book.

CHAPTER 2

Getting Started

Access, Pricing, and Your First Steps

Choosing Your Access Path

GPT-6 Astra is available through five primary channels. The right one for you depends on what you want to do, how much you want to spend, and whether you are building applications or working within ChatGPT.

Access Method	Best For	Starting Cost
ChatGPT Plus	Individual use, chat, Work	\$20/month
ChatGPT Pro	Power users, Astra Pro access	\$200/month
ChatGPT Business	Small teams	Per-seat pricing
ChatGPT Enterprise	Large organizations	Custom pricing
OpenAI API	Developers, custom apps	\$10/M input tokens
Azure / Bedrock	Enterprise cloud deployments	Per usage
GitHub Copilot	Software engineers	From \$10/month

ChatGPT Plans and Pricing

Astra is rolling out in phases. It is available to ChatGPT Plus, Pro, Business, and Enterprise users, as well as through the API. Astra usage is included within existing subscription allowances, and users can purchase additional credits for more usage. Pro, Business, and Enterprise subscribers also get access to GPT-6 Astra Pro, a higher-performance variant.

The Plus plan at \$20 per month is the most affordable way to access Astra. Access is gradual and initially reported in ChatGPT Work and Codex rather than the standard chat interface. Work is the productivity area where Astra can take on delegated tasks and complete multi-step workflows. Codex is

the coding environment where Astra can write features, fix bugs, run tests, and open pull requests.

Plus Plan Access Tip

If you have a Plus plan and do not see Astra yet, check ChatGPT Work and Codex sections in the desktop app. The rollout is gradual, and Astra appears in Work and Codex before it shows up in regular Chat. You can also use the desktop app for the best experience, as computer use features work there first.

API Access

For developers building custom applications, Astra is available through the OpenAI API with the model identifier "gpt-6-astra". The API supports both Chat Completions and Responses endpoints, but tool calling, structured outputs, streaming, computer use, and other advanced features require the Responses API.

Standard API pricing is \$10 per million input tokens and \$50 per million output tokens. Cached input costs \$1 per million tokens (a 90 percent discount), and cache writes cost \$12.50 per million tokens. Batch and Flex processing modes offer 50 percent off standard rates. Fast mode runs at 2x speed for 2x the price. These costs are covered in detail in Chapter 5.

Python: Your first API call

```
from openai import OpenAI
client = OpenAI() # uses OPENAI_API_KEY env var
response = client.responses.create(
    model="gpt-6-astra",
    reasoning={"effort": "medium"},
    input=[
        {"role": "developer", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain what GPT-6 Astra can do."}
    ]
)
print(response.output_text)
```

This minimal example uses the Responses API with medium reasoning effort. The reasoning parameter accepts five values: low, medium, high, xhigh, and max. Chapter 5 explains when to use each and how the choice affects both quality and cost.

Azure and AWS Bedrock

GPT-6 Astra is generally available on both Microsoft Azure AI Foundry and Amazon Bedrock. These platforms are the right choice for organizations that need enterprise security, compliance, and governance controls.

On Azure, Astra is available with Standard and Provisioned Throughput deployment options in both Global and US Data Zone geographies. Azure adds Microsoft Entra identity and access management, encryption in transit and at rest, private networking, role-based access controls, and content filtering.

On AWS Bedrock, Astra runs on Amazon's inference engine built for high performance, security, and scale. You can call the model directly through Bedrock APIs or configure ChatGPT Work and Codex to use Astra on Bedrock. Bedrock supports both implicit and explicit prompt caching to reduce costs.

Setting Up on Azure and AWS Bedrock: Step by Step

For organizations that need enterprise cloud deployment, here is how to get started on each platform.

Azure AI Foundry Setup

- 1 Create an Azure account if you do not have one, or sign in to the Azure portal.
- 2 Navigate to Azure AI Foundry (formerly Azure AI Studio).
- 3 Create a new project or open an existing one.
- 4 In the model catalog, search for GPT-6 Astra.
- 5 Choose a deployment option: Standard (pay-as-you-go) for variable demand, or Provisioned Throughput for dedicated capacity with consistent latency.
- 6 Select a geography: Global or US Data Zone, depending on your data residency requirements.
- 7 Deploy the model and note the endpoint URL and API key.
- 8 Call the model using the Azure SDK or REST API with your credentials.

Azure adds Microsoft Entra identity and access management, encryption in transit and at rest, private networking options, role-based access controls, content filtering, safety evaluations, monitoring, and governance tools on top of Astra's capabilities.

AWS Bedrock Setup

- 1 Sign in to the AWS Management Console.
- 2 Navigate to Amazon Bedrock.
- 3 In the left navigation, choose Model access and request access to GPT-6 Astra (OpenAI models section).
- 4 Once approved, go to Foundation models and find GPT-6 Astra.
- 5 Choose Open in playground to test the model, or Create agent to build an application.
- 6 For API access, use the Bedrock runtime API with your AWS credentials.

7 Configure prompt caching (implicit or explicit) based on your workload pattern.

Bedrock provides established AWS controls to secure workloads, govern access, and audit model invocation activity. You can call the model directly through Bedrock APIs or configure ChatGPT Work and Codex to use Astra on Bedrock.

Platform	Deployment Options	Data Residency	Best For
OpenAI API	Standard, Flex, Batch, Fast	Global	Developers, startups
Azure Foundry	Standard, Provisioned	Global, US Zone	Microsoft shops, enterprises
AWS Bedrock	On-demand, provisioned	AWS regions	AWS shops, enterprises
ChatGPT	Plus, Pro, Business, Enterprise	OpenAI-managed	Non-developers, teams

GitHub Copilot

GPT-6 Astra is available in GitHub Copilot for Copilot Pro+, Max, Business, and Enterprise users. You can select the model in the model picker across Visual Studio Code, Visual Studio, Copilot CLI, the GitHub Copilot coding agent, JetBrains IDEs, Xcode, Eclipse, and GitHub Mobile.

GitHub's internal testing found that Astra stands out for how it works, not just what it produces: it plans and validates as it goes, batches diagnosis with verification, and independently confirms its results before declaring a task done. This translated into stronger performance on long-horizon coding tasks with fewer steps than prior OpenAI models.

Your First API Call -- Detailed Walkthrough

If you have never used an API before, here is what each part of the code above does:

- 1 Install the OpenAI Python library by running "pip install openai" in your terminal or command prompt.
- 2 Get an API key from platform.openai.com by creating an account, adding billing information, and generating a new secret key.
- 3 Set your API key as an environment variable named OPENAI_API_KEY so the library can find it automatically.
- 4 Create a client object. The OpenAI() constructor reads your key from the environment.
- 5 Call client.responses.create() with the model name "gpt-6-astra", a reasoning effort level, and your input messages.
- 6 Print the output. The response object contains the generated text in the output_text field, along with token usage information.

Security Warning

Never put your API key directly in your code. Always use environment variables or a secrets manager. If you accidentally expose your key, revoke it immediately in the OpenAI dashboard. A leaked key can result in unauthorized charges to your account.

CHAPTER 3

Understanding Astra's Architecture

Context, Reasoning, Tools, and What Makes It Tick

The Million-Token Context Window

GPT-6 Astra has a context window of 1,050,000 tokens. In practical terms, that means you can send it roughly 790,000 words in a single request -- the equivalent of about ten full-length novels, an entire codebase, or hundreds of pages of legal documents. The model can maintain coherence across all of that material and retrieve specific details from anywhere in the input.

This is not just about length. It is about the kinds of work that become possible. You can give Astra an entire 200-page contract and ask it to identify every provision that carries risk. You can provide a complete codebase and ask it to trace a bug across multiple files. You can feed it a year of financial reports and ask for a reconciliation of conflicting data sources. Previous models lost track of details across long inputs. Astra does not.

The 272K Threshold

There is a critical pricing detail to know: prompts with more than 272,000 input tokens are billed at 2x the input and cache rates and 1.5x the output rate for the entire request. This means a single oversized prompt can double its own cost. Plan your context carefully. Chapter 5 covers strategies for staying under this threshold while keeping the context you need.

The maximum output is 128,000 tokens, which is roughly 96,000 words. This is enough for a substantial document, a detailed report, or a complete software module. Astra's training to complete tasks in fewer tokens means it often produces the same result in less output than previous models, which directly reduces cost.

Five Levels of Reasoning

Astra introduces five reasoning effort levels: low, medium, high, xhigh, and max. These control how much internal reasoning the model performs before producing its answer. Reasoning tokens count toward output usage, so higher effort means more cost as well as more thought.

Effort Level	When to Use	Relative Cost
low	Routine questions, simple lookups	Baseline
medium	Default starting point for most tasks	~1.5x low
high	Complex analysis, multi-step reasoning	~2x low
xhigh	Hard problems, debugging, planning	~2.5x low
max	Hardest tasks, research, architecture	~3.6x low

A third-party benchmarking service, Artificial Analysis, measured cost per Index task ranging from \$0.46 at low effort to \$1.67 at max effort -- a 3.6x swing. The intelligence gain from low to max is much smaller than the cost increase, which means max effort should be reserved for tasks where the additional reasoning genuinely improves the outcome.

The Golden Rule of Reasoning Effort

Start with medium. Test your task at low and see if the quality is acceptable. If it is, use low for routine instances. If you need better results, try high. Only escalate to xhigh or max when the task is genuinely difficult and the cost of a wrong answer is high. Astra's mid-conversation reasoning change feature (Chapter 5) lets you vary effort per step without losing your cached context.

Knowledge and Training

Astra's knowledge cutoff is April 30, 2026. This means the model's training data includes information up to that date. For anything after April 2026, Astra can use its built-in web search tool to find current information. Astra was pretrained at the Stargate facility -- what OpenAI researcher Aidan Clark described, in press reporting, as the company's largest training run ever.

External reporting suggests Astra may use a reasoning approach sometimes called "recurrent depth" or "looped transformers," said to increase efficiency while working in a way that can obscure some of the model's chain of thought. OpenAI has not officially confirmed this internal architecture in its public documentation. This has implications for monitoring and safety, which are discussed in Chapter 12. For most users, the practical effect is that Astra appears to deliver better results with fewer reasoning tokens.

How Astra Thinks: Reasoning Approach

External reporting indicates that Astra may use a reasoning technique called "recurrent depth" or "looped transformers," which is said to increase efficiency by reusing computation across reasoning steps. OpenAI has not officially confirmed or detailed this internal architecture in its public documentation. The discussion below is based on external reporting and Wikipedia's coverage; treat it as unconfirmed until OpenAI publishes technical details.

If the technique is in use, the practical effect for users would be two-fold. First, Astra would produce better results with fewer reasoning tokens, which directly reduces cost and latency. Second, the technique could work in a way that obscures some of the model's chain of thought -- OpenAI's safety documentation does note that Astra is more capable of controlling its own reasoning and less likely to include revealing information in it. This is why OpenAI deployed additional misalignment monitoring alongside the model.

You do not need to understand the internal architecture to benefit from the model. What matters is that you can control how much reasoning Astra does through the effort parameter, and that higher effort levels cost more because they involve more internal computation. The guidance is simple: start at medium, test at low, escalate when the task genuinely needs it.

The 512K to 1M Long-Context Test

On the 512K-to-1M-token long-context range, OpenAI reports 96.3 percent accuracy for Astra versus 73.8 percent for GPT-5.6 Sol. This means Astra can reliably find and use information buried deep in very long inputs -- a needle in a haystack test at scales that previous models could not handle. But remember: prompts above 272,000 tokens are billed at 2x rates. Use the capability when you need it; do not pad prompts with irrelevant context just because you can.

Input and Output Capabilities

The diagram below shows how Astra processes a request: input enters at the top, passes through reasoning, uses available tools, and produces output. Each layer is described in the sections that follow.

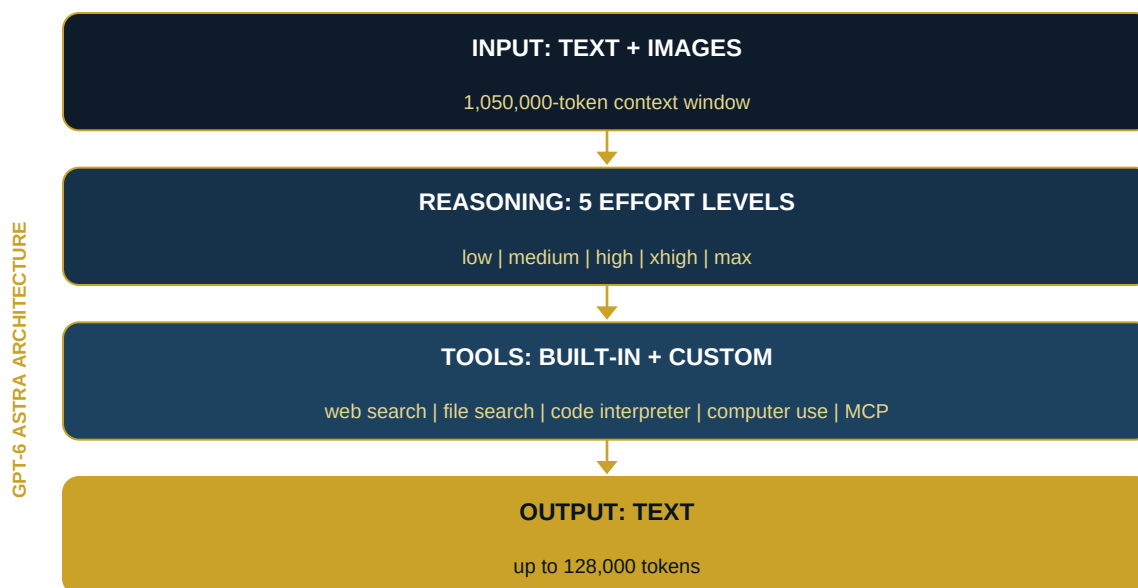


Figure 3.1 — Astra Architecture: from input to output

Astra accepts text and image input and produces text output. It does not generate images directly (though it can trigger image generation through its tools). It does not produce audio output natively. For images, it can analyze photographs, screenshots, diagrams, and documents -- which is essential for computer use, where it reads screenshots of the applications it is operating.

The model supports streaming, function calling, structured outputs, and all of OpenAI's hosted tools through the Responses API. Fine-tuning is not supported on the current model card. The model accepts text and image input and outputs text only.

The Responses API

The Responses API is the primary interface for building with Astra. While Chat Completions is supported for backward compatibility, the Responses API is required for tool calling, structured outputs, computer use, and other advanced features. OpenAI recommends it for all new integrations.

The Responses API introduces several features that Chat Completions lacks: asynchronous tool calling, mid-turn steering (redirecting the model partway through a response), persisted reasoning, compaction, and the ability to change reasoning effort mid-conversation without invalidating the prompt cache. These features are the building blocks of the agent systems and automation patterns covered in later chapters.

Chat Completions vs Responses

Use Chat Completions only when you have an existing integration that requires it. For all new work -- especially anything involving tools, agents, computer use, or structured data -- use the Responses API. It is the only way to access Astra's full capability set.

Feature	GPT-6 Astra	GPT-5.6 Sol	Claude Fable 5.1
Context window	1,050,000	Smaller	Smaller
Input price / 1M	\$10.00	\$4.00	\$10.00
Output price / 1M	\$50.00	\$20.00	\$50.00
Reasoning levels	5 (low-max)	Fewer	Fewer
Computer use	72.6% OSWorld	65.7% OSWorld	Not native
Terminal-Bench 4.0	57.9%	37.3%	55.8%
Computer-use safety	89% fewer errors	Baseline	74.7% fewer
Cyber classification	Critical	Below	Below

The comparison shows why Astra is priced at a premium. It matches Claude Fable 5.1 on sticker price but delivers measurably better computer use, terminal performance, and safety. Compared to GPT-5.6

Sol, it costs 2.5 times more per token but completes tasks in fewer tokens with fewer retries, often resulting in a lower total cost per task. The value is in the completion rate, not the sticker price.

Built-in Tools

Astra supports a comprehensive set of built-in tools through the Responses API. These tools are what transform Astra from a text generator into a system that can take actions and complete work:

Tool	What It Does
Web search	Searches the internet for current information
File search	Searches through uploaded documents
Image generation	Creates images through integrated tools
Code interpreter	Executes Python code to compute results
Hosted shell	Runs shell commands in a sandboxed environment
Apply patch	Applies code changes to files
Computer use	Operates browsers and desktop applications
Tool search	Discovers and uses available tools dynamically
MCP	Connects to external services via Model Context Protocol

These tools can be combined within a single workflow. For example, Astra can use web search to research a topic, code interpreter to analyze the data it finds, computer use to enter results into a spreadsheet, and file search to cross-reference with your existing documents. Chapter 6 covers computer use in detail, and Chapters 7 and 8 show how to combine tools in business workflows.

CHAPTER 4

Mastering Prompts

OpenAI's Official Framework for Getting the Best from Astra

Why Prompting Changed with Astra

If you have used GPT-4, GPT-5, or GPT-5.6 Sol, you will notice that Astra behaves differently. The most common reaction from experienced users is frustration: Astra keeps pausing to ask clarifying questions instead of just finishing the task. This is not a bug. It is a consequence of Astra's alignment training, which makes the model more likely to ask when additional input could materially change the result.

OpenAI published an official prompting guide for Astra that identifies five key behaviors to manage: initiative and follow-through, instruction following, personality and writing style, subagent delegation, and testing and verification. Mastering these five areas is the difference between an Astra that constantly stops to ask questions and one that powers through complex tasks to completion.

Initiative and Follow-Through

Astra is designed to be a more effective collaborator, which means it asks questions when it thinks the answer could change the outcome. This is generally good behavior, but it can be frustrating when you want the model to make reasonable assumptions and keep going. OpenAI provides specific prompts to control this behavior.

Prompt: Maximum Autonomy

You should infer the user's intent from context and proceed to completion. Your job is to bias towards action and deliver a concrete, reviewable result. Ask for approval only after preparing the work, not before starting. When the user's prompt indicates a request for action, assume authorization and proceed.

Prompt: Balanced Autonomy (Recommended Default)

When the user's prompt indicates a request for action, assume authorization and proceed. Infer routine implementation details. Ask for approval only before irreversible actions (deleting files, deploying, making payments) or when a missing requirement would materially change the result. Prepare a concrete, reviewable result before asking for review.

The key insight is that Astra respects boundaries very seriously. If you previously added strong language like "always ask before doing anything" to prevent earlier models from going too far, Astra will take that instruction literally and stop at every step. OpenAI recommends reviewing and softening boundary instructions because Astra has much better judgment about when to stop on its own.

Instruction Priority

Astra is better at following long instructions than previous models, but it is also more sensitive to instructions contained in skills, AGENTS.md files, and other contextual documents. When instructions conflict, Astra may pause and block work early rather than making a judgment call. The fix is to make priority explicit.

Prompt: Instruction Priority

The user's instructions take precedence over guidelines provided in a skill or reference document. If explicit user instructions conflict with a skill's instructions, prioritize the user's instructions. When you pause or change direction due to a conflicting instruction, name the specific skill and instruction that caused the pause.

This prompt solves a common failure mode: you give Astra a task, it loads a skill file that contains some cautious guidance, and it stops working to ask for clarification. By making priority explicit, you tell Astra that your direct instructions win over any loaded guidance. You also ask it to name the conflict, which gives you visibility into what is causing the pause so you can fix it.

Writing Style and Personality

Astra tends toward detailed, formatted responses and may use recurring phrases across sessions. If you do not specify a writing style, Astra will apply its default, which can feel generic or overly structured. OpenAI recommends showing the model your preferred tone through examples.

Prompt: Writing Style Control

Writing style: Use concise, direct prose. Avoid filler phrases like 'in conclusion,' 'it's worth noting,' 'leverage,' and 'it's important to understand.' Do not contrast unrequested alternatives ('this isn't X, it's Y'). Match the tone of the examples provided. Keep paragraphs under 5 lines.

For repeated work, OpenAI recommends saving your style preferences in a skill file that you reference every time. This is more consistent than re-stating the style in each prompt, especially in long conversations where earlier instructions can get diluted.

The Prompt Structure Framework

OpenAI's model guidance defines a framework for structuring prompts that work reliably with Astra. A well-structured prompt includes seven components. Not every prompt needs all seven, but the more complex the task, the more important they become. The diagram below shows the full stack:

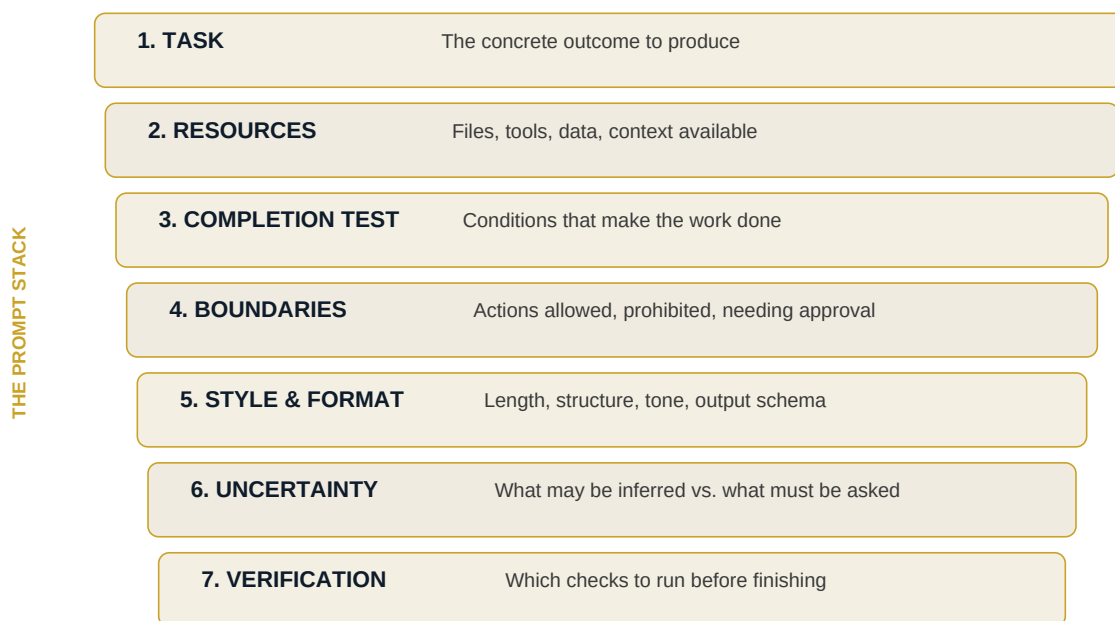


Figure 4.1 — The seven-component prompt stack

Component	What to Specify
Task	The concrete outcome to produce
Resources	Files, tools, data, and context available

Component	What to Specify
Completion test	Conditions that make the work done
Boundaries	Actions allowed, prohibited, or needing approval
Style and format	Length, structure, tone, output schema
Uncertainty	What may be inferred and what must be clarified
Verification	Which checks are required and when to stop

Complete Prompt Example: API Design Review

Task: Review this API design and identify the three highest-impact migration risks.

Resources: Use the attached schema and deployment notes.

Completion test: Return three risks, evidence for each, and one acceptance test per risk.

Boundaries: Do not change production systems. Infer routine implementation details.

Clarification rule: Ask only if a missing requirement would materially change the result.

Style: Use concise prose and a final three-row table.

Verification: Check that every risk has an executable acceptance test.

Notice how each component of the framework appears in this prompt. The task is specific (three risks, not "review everything"). The completion test is concrete (three risks with evidence and tests). The boundaries prevent dangerous actions. The clarification rule tells Astra when to ask versus when to infer. And the verification step tells Astra how to self-check before declaring the task done.

Subagent Delegation

Astra can divide work across subagents, but OpenAI notes it may delegate less often than some multi-agent frameworks expect unless the prompt explains when delegation is useful. If you want Astra to parallelize independent work, say so explicitly.

Prompt: Enabling Delegation

When a task contains independent sub-tasks that can run in parallel, delegate each to a subagent. Specify: (1) what each subagent should do, (2) what resources it has access to, (3) what completion criteria it should meet, and (4) when results should be combined. Do not run independent sub-tasks sequentially when they could run in parallel.

Delegation is most valuable for tasks with clearly independent parts: researching multiple topics in parallel, testing different approaches to a problem, or processing separate data sets. It is less useful for tightly coupled tasks where each step depends on the previous one. Over-delegation adds overhead without benefit.

Testing and Verification

Astra is thorough about verification -- sometimes too thorough. Previous models needed encouragement to run tests and check their work. Astra does that on its own, so instructions that tell it to test can lead to unnecessary testing of trivial changes. If you find Astra running the full test suite for a one-line fix, adjust your instructions to specify when testing is appropriate.

Calibrating Verification Scope

The goal is verification proportional to risk. A one-line typo fix does not need the full test suite. A change to a payment processing function does. Tell Astra: 'Verify at a level proportional to the risk of the change. For minor changes, verify only the affected code. For significant changes, run the full test suite.' This saves tokens (and therefore money) while maintaining quality.

For coding work, OpenAI recommends defining what 'done' means before starting. If the task includes getting the implementation running, inspecting the result, and fixing what fails, make that part of the request. Otherwise, Astra may reach a first implementation and stop for review while there is still work to do.

Copy-Paste Prompt Templates

Here are five ready-to-use prompt templates for common tasks. Customize the bracketed sections for your specific needs.

Template 1: Research and Report

Research Template

Task: Research [topic] and produce a structured report.

Resources: Use web search. Focus on sources from [last 12 months].

Completion test: A 2-page report with an executive summary, 3-5 key findings with evidence, and a recommendations section.

Boundaries: Cite all sources. Do not fabricate data or quotes.

Style: Professional, concise, data-driven. Use headers and bullets.

Verification: Confirm every claim has a cited source.

Template 2: Code Generation

Code Generation Template

Task: Write a [Python/JavaScript] function that [description].

Resources: Use the attached [file/specification].

Completion test: Working code with type hints, docstring, and at least 3 test cases covering normal, edge, and error cases.

Boundaries: Do not modify existing code outside the target function.

Style: Follow [PEP 8 / Standard JS]. Use descriptive variable names.

Verification: Run the tests mentally and confirm they pass.

Template 3: Document Creation

Document Template

Task: Create a [proposal/memo/report] for [audience] about [topic].

Resources: Use the provided [brief/template/data].

Completion test: A polished [N]-page document following the template structure, ready for review without reformatting.

Boundaries: Follow the template exactly. Do not add sections not in the template.

Style: Match [our company voice / formal / conversational].

Verification: Check that all template sections are present and filled.

Template 4: Data Analysis

Analysis Template

Task: Analyze the attached [dataset/spreadsheet] and identify [trends/anomalies/opportunities].

Resources: Use code interpreter to process the data.

Completion test: A summary with 3-5 findings, supporting calculations, and one visualization.

Boundaries: Do not modify the original data. Work on a copy.

Style: Data-driven prose. Lead with numbers, then explanation.

Verification: Recalculate key figures to confirm accuracy.

Template 5: Computer Use Automation

Computer Use Template

Task: [Navigate to website X, fill form Y, extract data Z].

Resources: Use the computer use tool. The target URL is [URL].

Completion test: The form is submitted and a confirmation page is visible, or the requested data is extracted into a table.

Boundaries: Do not make purchases, submit payments, or delete data without explicit approval.

Verification: Take a screenshot of the final state and confirm the expected result is visible.

Pro Tip: Define Completion Before Starting

Astra may reach a first implementation and come back for your review while there is still work to do. If the task includes getting the implementation running, inspecting the result, and fixing what fails, make that part of the request. A requirement to "stop for review after the first implementation" will pull the model toward an earlier stopping point than you want.

Advanced Prompting Techniques

Few-Shot Prompting with Style Examples

Astra learns your preferred tone fastest from examples. Instead of describing your style, show it. Attach 2-3 examples of your best writing and ask Astra to match them. This works better than any style description because the model can infer patterns you cannot articulate.

Style Matching Prompt

I've attached three examples of my writing. Study their tone, sentence structure, and vocabulary. Then write [new content] in the same style. Do not copy phrases from the examples -- match the style, not the words. If my style has specific patterns (short sentences, active voice, direct address), maintain them.

Chain-of-Thought Direction

While Astra reasons internally, you can direct its reasoning process for complex problems. Instead of asking for an answer, ask for the reasoning that leads to the answer. This produces better results on problems where the process matters as much as the conclusion.

Reasoning Direction Prompt

Before giving your answer, work through the problem in this order:

1. What are we actually being asked to decide?
2. What information do we have, and what is missing?
3. What are the three most plausible approaches?
4. What are the tradeoffs of each?
5. Which approach best fits our constraints, and why?

Then give your recommendation with your confidence level and the single most important piece of missing information.

Self-Critique and Iteration

Astra can review its own output if you ask it to. After generating a deliverable, ask for a critique pass: identify the weakest points, the claims with the least support, and the sections a critical reviewer would challenge. Then ask for a revision that addresses those weaknesses.

Self-Critique Prompt

Review your previous output as a critical expert reviewer would. Identify: (1) the weakest argument and why, (2) any claim that needs more evidence, (3) anything a knowledgeable reader would question, (4) what is missing that should be included. Then revise the output to address each weakness. Present both the critique and the revision.

Context Priming

For specialized work, prime Astra with domain context before the task. A brief primer -- key terms, relevant constraints, important stakeholders -- improves output quality without adding much cost. Keep the primer stable across requests so it benefits from prompt caching.

Context Priming Prompt

Context before the task: We are a [type of company] serving [audience]. Our key differentiator is [differentiator]. We measure success by [metrics]. Our competitors are [competitors]. Our brand voice is [voice description].
Now, [the actual task].

Keep the Primer Cache-Friendly

Because the primer is stable across requests, put it at the very beginning of your prompt. This lets prompt caching reuse it at a tenth of the cost. If the primer is 2,000 tokens and you make 100 requests, caching saves you \$17.80 in input costs alone. For production workflows, primers pay for themselves many times over.

CHAPTER 5

Slashing Costs

How to Use Astra Without Breaking the Bank

Understanding the Pricing Model

Astra's sticker price is \$10 per million input tokens and \$50 per million output tokens. That is 2.5 times what GPT-5.6 Sol charges per token. But the sticker price is the least useful number for understanding your actual bill. What you really pay depends on three levers that most pricing pages do not emphasize: your reasoning effort setting, whether your prompt crosses the long-context threshold, and how well you reuse cached input.

Token Type	Price per 1M Tokens	Notes
Input (standard)	\$10.00	Fresh input tokens
Cached input	\$1.00	90% discount on reused prefixes
Cache writes	\$12.50	1.25x input rate, paid once
Output	\$50.00	Generated text
Batch / Flex	50% off	Non-urgent workloads
Fast mode	2x standard	Double speed, double price
Long context (>272K)	2x input, 1.5x output	Entire request

Two details in this table carry real weight. First, cached input costs a tenth of fresh input. Anything you send repeatedly should be cached. Second, cache writes cost 1.25 times the uncached input rate -- a small premium you pay once to unlock the 90 percent discount on every later read. The break-even is immediate: one write and one read costs 1.35x the input, compared to 2x for processing the same prefix twice without caching.

The 272K Threshold Trap

When a request exceeds 272,000 input tokens, OpenAI bills the entire request at 2 times the input and cache rates and 1.5 times the output rate. Not just the portion above 272K -- the entire request. A single oversized prompt can quietly double its own input cost, and since the surcharge applies to output as well, the effect compounds.

The 272K Trap in Practice

Suppose you send a 280,000-token prompt (a large codebase plus instructions). Instead of paying \$2.80 for input at the standard rate, you pay \$5.60 because the entire request is billed at 2x. Your output, which would cost \$0.50 at standard rates for 10,000 tokens, now costs \$0.75. The difference between 270K and 280K input tokens is not a 3.7 percent increase -- it is roughly a 100 percent increase in total cost.

The strategies to avoid this trap are straightforward: retrieve only the documents needed for the current step, use file search instead of embedding everything in the prompt, split large requests into smaller ones, and use compaction for long-running conversations. Chapter 11 covers these techniques in depth.

Prompt Caching: Your Biggest Lever

Prompt caching is the single most effective way to lower your Astra bill. It reduces input-token billing by up to 90 percent and cuts time-to-first-token by up to 80 percent. It works automatically on any prompt over 1,024 tokens, with no code changes required. But to get the maximum benefit, you need to structure your prompts correctly.

The cache is prefix-based. OpenAI reuses tokens from the beginning of a prompt when the next request starts with matching content. Think of your prompt as a document: stable chapters come first, request-specific appendix comes last. If you put stable content (system instructions, tool definitions, reference material) at the front and variable content (user messages, search results) at the back, the stable prefix can be cached and reused across requests at a tenth of the cost.

Third-Party Example: 92 Percent Cost Reduction

ComputingForGeeks, an independent tech site, tested prompt caching by sending the same 22,500-token configuration file to Astra twice. The first (cold) call cost \$0.28. The second (warm) call, with an identical prefix, cost \$0.023 -- a 91.9 percent reduction for byte-identical input. This is a third-party test, not an OpenAI-published result, but it illustrates the caching mechanism. If your workload reuses a large context, structure it so the stable part sits at the front of every request.

Implicit vs Explicit Caching

Astra supports two caching modes. In implicit mode (the default), OpenAI places a cache breakpoint automatically. This is the easiest starting point and works well when your prompt has one large, stable prefix.

Explicit mode gives you more control. You add `prompt_cache_breakpoint` content items at boundaries worth preserving. This is useful when your prompt is assembled from several stable blocks followed by volatile material. A request can use at most four breakpoints, and the service considers up to the latest 80 breakpoints across a conversation.

The Default Trap

On GPT-5.6 and later (including Astra), the implicit breakpoint lands on the latest user or tool message. If that message contains a timestamp, a fresh user input, or any changing content, the prefix hash differs every time. The result: `cached_tokens` can be zero while the service writes the changing prefix again at 1.25x cost. This turns a workload that should cost 0.1x into one costing 1.25x -- a 25 percent surcharge for a feature that should save you 90 percent. Fix it by setting `prompt_cache_options.mode` to 'explicit' and placing your breakpoint after the stable content.

Python: Explicit cache configuration

```
response = client.responses.create(
    model="gpt-6-astra",
    input=[
        {
            "role": "developer",
            "content": [
                {
                    "type": "input_text",
                    "text": "You are a financial analyst. Follow these rules: ...",
                    "prompt_cache_breakpoint": {"mode": "explicit"}
                }
            ],
        },
        {
            "role": "user",
            "content": "Analyze this quarter's data: ..." # volatile, after breakpoint
        }
    ],
    prompt_cache_options={"mode": "explicit", "ttl": "30m"}
)
```

Cache Best Practices

The chart below shows how caching savings compound as you make more requests with the same prefix. At 100 requests, caching costs 11.15x compared to 100x without it — an 89 percent reduction:

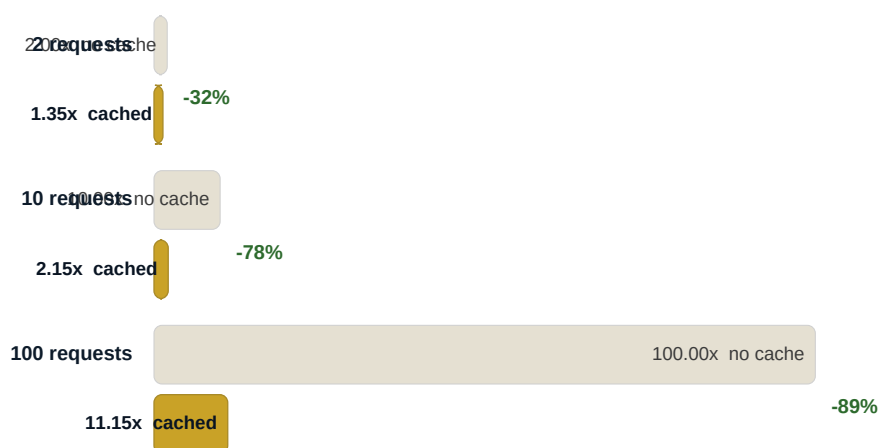


Figure 5.2 — Cache savings: cached vs uncached cost multiplier

- **Stable first, volatile last.** Put system instructions, tool schemas, and reference documents at the beginning. Put user input, search results, and temporary state at the end.
- **Keep tool schemas stable.** Tool names, descriptions, properties, and ordering must be identical across requests. Remove unused tools to lower both cached and uncached context.
- **Use `prompt_cache_key`.** On GPT-5.6 and later, OpenAI recommends setting this parameter for more reliable cache matching. It is optional but improves hit rates. Keep traffic per key to approximately 15 requests per minute.
- **Monitor `cached_tokens`.** Check `usage.input_tokens_details.cached_tokens`. If it is zero on second and subsequent requests, your prefix is drifting.
- **Version deliberately.** A policy edit should create a new cache key, not silently mix measurements from two versions.

Reasoning Effort Optimization

Reasoning effort is the biggest cost dial after caching. Artificial Analysis found that cost per Index task ranges from \$0.46 at low effort to \$1.67 at max effort -- a 3.6x swing. But the intelligence gain from low to max is much smaller than the cost increase.

The strategy is simple: use the lowest effort that passes your quality bar. Test your task at low first. If the output is acceptable, use low for routine instances. Reserve higher effort for genuinely difficult problems where the extra reasoning improves the outcome enough to justify the cost.

Task Type	Recommended Effort	Why
Simple Q&A, lookups	low	Minimal reasoning needed
Standard analysis, drafting	medium	Balanced cost/quality

Task Type	Recommended Effort	Why
Complex debugging, planning	high	Extra reasoning helps
Architecture, research	xhigh	Hard problems, high stakes
Frontier problems	max	Spare no expense

Mid-Conversation Reasoning Changes

One of Astra's most valuable cost features is the ability to change reasoning effort mid-conversation without invalidating the prompt cache. The key detail from OpenAI's documentation: keep the request-level `reasoning.effort` parameter unchanged across requests, because request-level changes can alter reasoning instructions and break prefix matching. Instead, append a `configuration_update` input item to the conversation when you want more or less reasoning for a specific stretch of work. The updated effort applies until another `configuration_update` overrides it, and the original prefix stays eligible for cache reuse.

The result: you can run cheap shallow reasoning across most of a task and turn it up only for the steps that need it. Hit a hard problem? Send a `configuration_update` raising effort to high or xhigh. Back to an easy task? Drop it to low. The long context from earlier stays reusable. This is a structural cost advantage for long agent sessions, not a marginal optimization.

Python: Changing reasoning mid-conversation (cache-preserving)

```
# Start with low effort for routine work
response1 = client.responses.create(
    model="gpt-6-astra",
    reasoning={"effort": "low"},
    input=[{"role": "user", "content": "Summarize this document..."}]
)

# Raise effort mid-conversation WITHOUT touching the request-level
# reasoning parameter -- append a configuration_update input item:
response2 = client.responses.create(
    model="gpt-6-astra",
    reasoning={"effort": "low"}, # keep unchanged for cache reuse
    previous_response_id=response1.id,
    input=[
        {"type": "configuration_update",
         "reasoning": {"effort": "high"}},
        {"role": "user", "content": "Find the critical security flaw..."}
    ]
)

# The cached prefix from response1 is still valid.
```

Batch and Flex Modes

If your workload does not need real-time responses, Batch and Flex modes offer 50 percent off standard rates. Batch processes requests asynchronously with a 24-hour turnaround. Flex runs through the Responses API with `service_tier` set to "flex" and offers the same 50 percent discount with more flexibility.

The official OpenAI Cookbook ran a head-to-head test of 10,000 identical requests and found that Flex with extended prompt caching and `prompt_cache_key` produced an 8.5 percent increase in cache hit rate compared to the Batch API, translating into a 23 percent reduction in input token cost. For latency-insensitive workloads with stable prefixes, Flex with caching is often the best option.

Mode	Discount	Turnaround	Best For
Standard	None	Real-time	Interactive applications
Flex	50% off	Near real-time	Latency-tolerant workloads
Batch	50% off	Up to 24 hours	Bulk processing
Fast	+100% (2x)	2.5x faster	Time-critical tasks

The Cache Economics, in Detail

Understanding the exact economics of prompt caching helps you make better architectural decisions. Here is the math that matters:

- 1 A cache write costs 1.25x the normal input rate. For 1 million tokens, that is \$12.50 instead of \$10.00.
- 2 A cache read costs 0.1x the normal input rate. For 1 million tokens, that is \$1.00 instead of \$10.00 -- a 90 percent discount.
- 3 Writing a prefix once and reusing it once costs 1.35x (1.25 write + 0.1 read) of its ordinary input cost, compared to 2x for processing it twice without caching. You save 32.5 percent on the second use alone.
- 4 Across 10 requests, one write and 9 reads cost 2.15x (1.25 + 9 x 0.1), compared to 10x without caching. That is a 78.5 percent savings.
- 5 Across 100 requests, one write and 99 reads cost 11.15x (1.25 + 99 x 0.1), compared to 100x. That is an 88.85 percent savings.

The break-even point is the second request. From the second cache read onward, every additional read is pure savings. If your workflow involves sending the same prefix more than twice, caching pays for itself.

The Cache TTL

Cached prefixes are retained for at least 30 minutes after their most recent write or reuse. OpenAI may retain them longer. Reusing a cached prefix refreshes its lifetime without another write charge. This means a workload with requests every few minutes keeps the cache continuously warm, while a workload with gaps of an hour or more pays to re-write the prefix each time. Plan your request patterns accordingly.

Model Routing Strategy

Not every task needs Astra. One of the most effective cost strategies is to route simple traffic to a cheaper model and reserve Astra for the hard 5 percent of requests where its capabilities genuinely matter. OpenAI itself says Astra is most valuable when an API call is one step in a difficult workflow rather than the end of one.

Model Routing Principle

Use Astra when its additional capability is justified by task difficulty, tool use, long context, reliability requirements, or the cost of failure. Use cheaper models when they meet the required quality level. Measure quality, latency, and cost for your own workload before choosing a routing policy -- the right split between models depends entirely on what your traffic looks like.

Tasks that justify Astra's higher per-token cost include: repository-scale debugging and migration, browser or computer-use agents, deep research across multiple sources, very long document or codebase analysis, scientific workflows, and professional automation where a failed run creates meaningful recovery cost. Tasks that do not need Astra include: simple Q&A, single-file edits, basic text generation, routine lookups, and anything where a cheaper model produces equally good output.

Real Cost Calculations

Here is a worked example to show how the pieces fit together. Suppose you have a task that sends 50,000 input tokens and receives 10,000 output tokens at high effort. The chart below shows how each optimization reduces the cost:

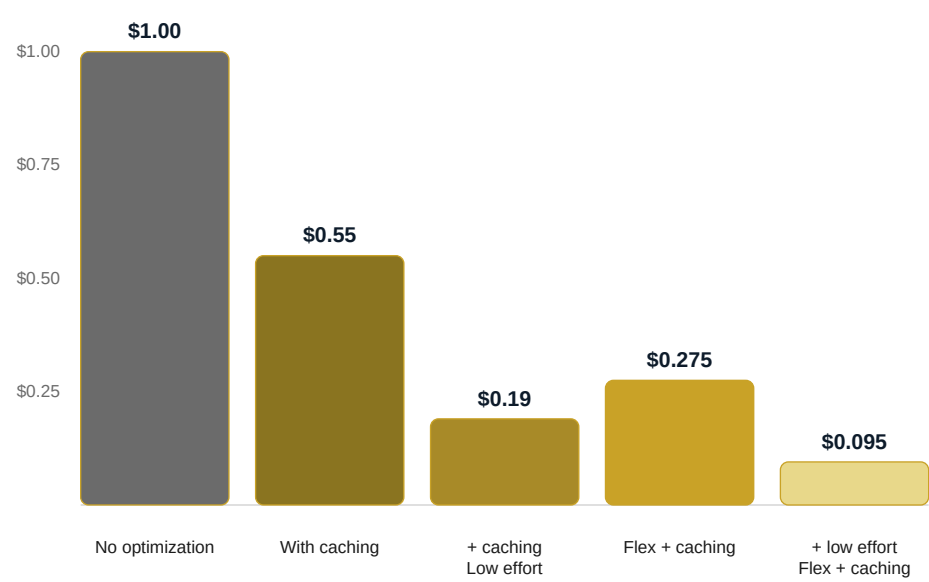


Figure 5.1 — Cost per task (\$) with each optimization layer applied

Scenario	Input Cost	Output Cost	Total	Savings
No optimization	\$0.50	\$0.50	\$1.00	Baseline
With caching (90%)	\$0.05	\$0.50	\$0.55	45%
Low effort + caching	\$0.05	\$0.14	\$0.19	81%
Flex + caching	\$0.025	\$0.25	\$0.275	72%
Flex + caching + low	\$0.025	\$0.07	\$0.095	90.5%

Run that task 1,000 times a day and the difference between no optimization (\$1,000/day) and full optimization (\$95/day) is \$905 per day, or roughly \$27,000 per month. That is the difference between a viable business and an unsustainable one.

Cost Optimization Checklist

1. Structure prompts with stable content first, volatile content last.
2. Set `prompt_cache_key` for reliable cache matching.
3. Use explicit mode to avoid the implicit breakpoint trap.
4. Start reasoning at medium, test at low.
5. Change effort mid-conversation for hard steps.
6. Keep prompts under 272K tokens.
7. Use Flex or Batch for non-urgent work.
8. Route simple tasks to cheaper models.
9. Monitor `cached_tokens` and `cache_write_tokens`.
10. Measure cost per accepted task, not cost per token.

Your Cost Calculator

Fill in this worksheet for your own workload. All pricing uses the official Astra rates: \$10 per 1M input tokens, \$1 per 1M cached input tokens, and \$50 per 1M output tokens. Work through the rows in order; the final row gives your estimated monthly bill before and after optimization. These are estimates -- actual costs depend on your cache hit rate and reasoning effort.

Step	Fill In	Your Value
A. Average input tokens per request	from your logs	_____
B. Average output tokens per request	from your logs	_____
C. Requests per day	your volume	_____
D. Baseline daily cost = $(A \times 10 + B \times 50) / 1,000,000 \times C$	dollars/day	_____
E. Share of input tokens that repeat (0-100%)	your estimate	_____
F. Cached savings = $A \times E\% \times 9 / 1,000,000 \times C$	dollars/day saved	_____
G. Flex/Batch eligible share of requests (0-100%)	non-urgent work	_____
H. Flex savings = $D \times G\% \times 0.5$	dollars/day saved	_____
I. Optimized daily cost = $D - F - H$	dollars/day	_____

Step	Fill In	Your Value
J. Monthly estimate = I x 30	dollars/month	_____

Worked Example

A workload with A = 20,000 input tokens, B = 2,000 output tokens, and C = 200 requests/day has a baseline of $(20,000 \times 10 + 2,000 \times 50) / 1,000,000 \times 200 = \$60/\text{day}$. If 80 percent of input tokens repeat (E = 80%), caching saves \$28.80/day. If half the requests can run on Flex (G = 50%), Flex saves another \$15.00/day. The optimized bill is \$16.20/day or about \$486/month -- down from \$1,800/month. This is an illustrative calculation; verify against your actual usage dashboard.

CHAPTER 6

Computer Use and Browser Automation

Making Astra Operate Your Software

What Is Computer Use?

Computer use is Astra's ability to reason about a visible interface and propose interactions: navigating, clicking, typing, scrolling, and reading the next screen. It is what allows Astra to fill out online forms, update customer records in a CRM, organize calendars, build websites, run QA checks, and work through applications that do not have developer APIs.

This is the capability that separates Astra from every previous generation of AI. Earlier models could tell you how to do something on a computer. Astra can do it. OpenAI's launch demonstration showed Astra navigating from Google Maps to a pediatrician's website and submitting a contact form, listing items on eBay, building a 3D game in Blender, and ordering food -- all by interacting with the same interfaces a human would use.

The Key Distinction

Traditional automation follows fixed rules: if X happens, do Y. Astra's computer use is flexible: it examines the situation, chooses an appropriate action, and adjusts its approach when new information appears. This means it can handle workflows involving multiple tools, changing interfaces, and unexpected conditions that would break a scripted automation.

The Observe-Decide-Execute Loop

Computer use works in a loop. Astra observes the current screen (via a screenshot), decides what action to take, executes that action through your application, and then observes the result. The loop repeats until the task is complete or a stopping condition is met. The diagram below shows this cycle:

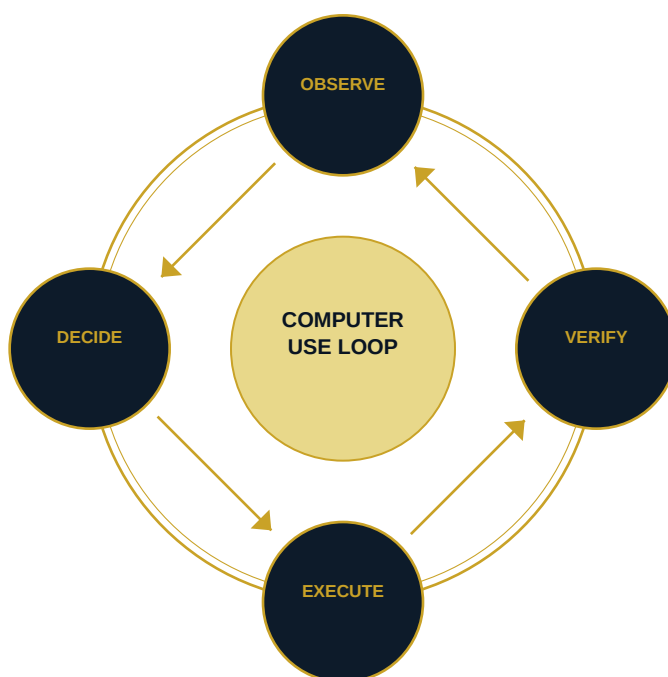


Figure 6.1 — The computer-use loop: observe, decide, execute, verify

- 1 **Observe:** Astra receives a screenshot of the current screen state and any available structured context (URL, page title, element attributes).
- 2 **Decide:** The model reasons about what it sees and determines the next action: click a button, type text, navigate to a URL, scroll, or report completion.
- 3 **Execute:** Your application carries out the requested action in the controlled environment and captures a new screenshot.
- 4 **Verify:** Astra examines the new screenshot to confirm the action had the expected effect. If not, it adjusts and tries again.

On OSWorld 2.0, which tests computer use on real desktop environments, Astra scores 72.6 percent and completes tasks in about 40 minutes per task, compared to 65.7 percent in 75 minutes for GPT-5.6 Sol. Combined with the updated Codex harness, Astra completes tasks 1.9 times faster than the previous experience on the Mind2Web benchmark.

Code Execution vs Structured Tool

OpenAI documents two approaches to computer use with Astra. The recommended path is code execution: Astra writes a Playwright or PyAutoGUI script for a function you provide. The native "computer" tool remains supported when an application already expects structured mouse and keyboard actions.

Code execution is preferred because it can reduce round trips -- the model can inspect a page, perform a short sequence, and report the updated state in one script. It can also use Playwright locators when coordinates would be brittle. The structured computer tool is better when your control plane is built around a fixed action vocabulary or your audit trail needs every requested input action.

Approach	Best For	How It Works
Code execution	New integrations	Astra writes Playwright/PyAutoGUI scripts
Structured tool	Existing control planes	Astra requests specific actions (click, type, scroll)

Building a Computer Use Workflow

A production computer-use workflow requires seven components. Each serves a specific safety and reliability function:

- 1 **Task service:** Receives the user's goal and defines the allowed scope.
- 2 **Policy engine:** Checks domains, action types, and confirmation requirements.
- 3 **Model client:** Calls Astra through the Responses API with the computer-use tool.
- 4 **Isolated executor:** Runs Playwright, PyAutoGUI, or structured actions in a sandboxed browser or VM.
- 5 **Observation service:** Captures screenshots, URLs, logs, and assertions.
- 6 **Approval interface:** Asks a human to confirm consequential steps.
- 7 **Audit store:** Records decisions, actions, results, and final verification.

A Complete Computer Use Example

Here is a concrete walkthrough of a computer-use workflow for a common business task: monitoring a competitor's website for pricing changes and recording the results in a spreadsheet.

Competitor Monitoring Prompt

Task: Visit [competitor website] and record their current pricing for [product category]. Compare with our pricing in the attached spreadsheet. Flag any products where their price is more than 10% below ours.

Resources: Use computer use to navigate the website. Use the attached spreadsheet for our current pricing.

Completion test: A table with product name, their price, our price, difference, and a flag for items more than 10% below ours.

Boundaries: Read-only. Do not interact with their checkout.

Do not create accounts or submit any forms.

Verification: Confirm each price was read from the live page, not from cached search results.

This workflow demonstrates several computer-use principles. It is read-only, which means the worst-case failure is a wrong number, not a destructive action. It has a clear completion test. It specifies boundaries (no checkout interaction, no account creation). And it includes a verification step that guards against the most common failure: reading stale cached data instead of live page content.

Real-World Examples from the Launch

OpenAI's launch demonstration showed the breadth of what Astra can do in a single session. These examples, drawn from the official launch video and announcement, illustrate the range:

- **Design to 3D:** Draw a small yellow circle in an image editor, turn it into the window of a rocket ship, add detail, then convert it to a 3D model in Blender and export an STL file for 3D printing.
- **Presentation creation:** Build a high-end, colorful presentation for next season's rainwear for retailers, matching the brand's visual style and adjusting background colors to complement the products.
- **Marketplace listing:** Navigate to eBay, create a listing for a used table from a photo in the downloads folder, note the slight dent in the description, and post it for sale.
- **Game development:** Build a playable 3D game where the user dodges asteroids using arrow keys, with a boost on space.
- **Legal drafting:** Generate a licensing agreement template for lawyers at a firm to review, then tighten the limitation of liability provision to be more favorable to the licensor.
- **Life logistics:** Find a tennis court in a specific neighborhood, check availability, and book it for the afternoon.

- **Food ordering:** Order beef and rice from a restaurant the user had ordered from previously.

What is notable about these examples is not any single task -- earlier models could describe how to do each one. What is new is that Astra actually does them: navigating real interfaces, making real changes, and completing multi-step workflows without a human prompting each step. The gap between "AI that tells you how" and "AI that does it" has closed.

Safety Controls

Computer use gives Astra the ability to affect the real world through software, which means safety controls are not optional. OpenAI's guidance places controls in the application and execution environment, not just in model instructions.

- **Use an isolated environment.** A separate browser profile, container, or VM with the smallest useful permissions. A testing agent checking a landing page does not need access to email, cloud drives, or production admin panels.
- **Restrict navigation.** Limit navigation to expected domains and block redirects to unapproved origins.
- **Allow specific action classes.** Allow reading, clicking, and typing test data. Disallow downloads, uploads, or clipboard access unless required.
- **Require just-in-time approval.** Require human confirmation for consequential actions: payments, data deletion, publication, account changes, or irreversible messages.
- **Set limits.** Cap steps, time, cost, and retries. Define a stopping condition before the task begins.
- **Treat screenshots as untrusted.** A webpage can contain prompt injection asking the model to reveal credentials or change goals. Validate structured state independently.
- **Verify completion from observable state.** Use the most authoritative observation available: a fresh screenshot, a structured read API, a database record, or an application audit event.

The Confirmation Policy Is Non-Negotiable

Never run computer use without a written confirmation policy and a human who can approve. OpenAI's own safety numbers show the misaligned-outcome rate in adversarial workplace tests fell from 3.4 percent without a policy to 3.0 percent with one. Unauthorized external communications fell from 1.7 percent to 1.4 percent. For GPT-5.6 Sol, the same tests showed 18.8 percent and 8.0 percent respectively. The policy matters.

Enterprise Controls

Organizations can control how broadly Astra is deployed. New enterprise admin controls let administrators restrict access to approved websites and desktop applications, manage uploads and downloads, and control browsing history. ChatGPT Work and Codex include confirmation policies that require approval before consequential actions and automated review of potentially unsafe or unauthorized tool calls.

Enterprise plugins extend Astra's browser-use capabilities across common business applications. Plugins from Oracle Analytics, Power BI, Navan, and Avalara make it easier to access familiar enterprise tools through ChatGPT Desktop. These plugins work through existing user accounts and within administrator-established permissions -- they do not grant Astra additional access.

CHAPTER 7

Building Your First AI Agent

From Concept to Working Code

What Is an AI Agent?

An AI agent is a system that can pursue a goal across multiple steps, using tools and making decisions along the way, without requiring a human to prompt it at each step. Unlike a chatbot that responds to one message at a time, an agent takes a goal, breaks it down, executes the steps, calls tools, evaluates results, and either completes the task or asks for help when it genuinely cannot proceed.

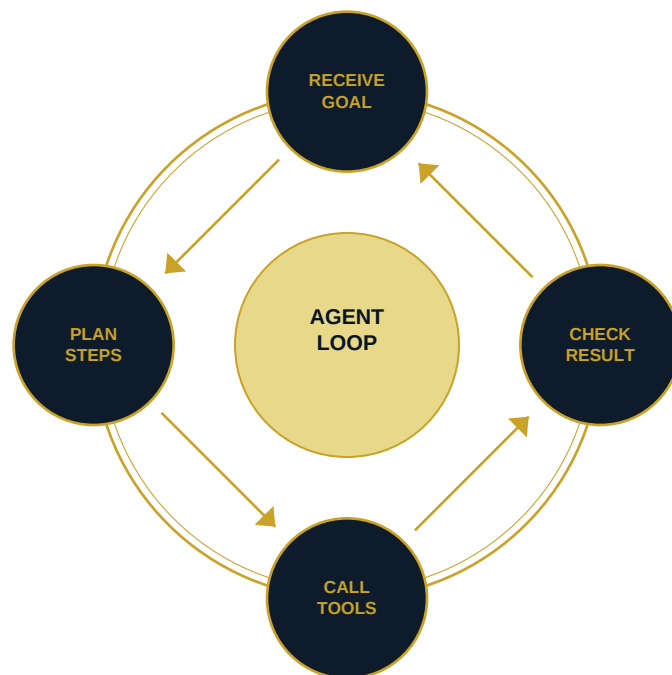


Figure 7.1 — The agent loop: goal to plan to tools to verification

Astra is designed for exactly this kind of work. Its strengths -- long-horizon reasoning, tool use, computer interaction, and context management -- are the building blocks of agents. OpenAI's benchmarks specifically measure agentic performance: Agents' Last Exam (59.3 percent), AutomationBench (41.4 percent), and Terminal-Bench 4.0 (57.9 percent).

The Responses API

The Responses API is the foundation for building agents with Astra. It supports function calling, structured outputs, streaming, persisted reasoning, compaction, and tool use. The key difference from Chat Completions is that the Responses API maintains state across turns: you can chain responses using `previous_response_id`, and the model retains context without you re-sending the full conversation history.

Python: Basic agent loop

```
from openai import OpenAI
import json
client = OpenAI()
# Define a tool the agent can call
tools = [{
    'type': 'function',
    'name': 'get_weather',
    'description': 'Get current weather for a city',
    'parameters': {
        'type': 'object',
        'properties': {
            'city': {'type': 'string'}
        },
        'required': ['city']
    }
}]
# Send the task
response = client.responses.create(
    model="gpt-6-astra",
    reasoning={"effort": "medium"},
    tools=tools,
    input="Should I carry an umbrella in Mumbai today?"
)
# Check if the model wants to call a tool
for item in response.output:
    if item.type == 'function_call':
        args = json.loads(item.arguments)
        result = get_weather(**args) # your function
# Send the result back
response2 = client.responses.create(
    model="gpt-6-astra",
    previous_response_id=response.id,
    input=[{
        'type': 'function_call_output',
        'call_id': item.call_id,
        'output': json.dumps(result)
    }]
)
print(response2.output_text)
```

Function Calling

Function calling is how agents take actions. You define functions (also called tools) with a name, description, and parameter schema. Astra decides when to call a function, proposes arguments, and your application executes the function. The model proposes arguments; it does not grant permission to perform the action. Your application decides whether to execute, with its own authorization, validation, and confirmation boundaries.

The Authorization Rule

Schema validation is not authorization. Even if Astra proposes valid arguments for a function, your application must verify that the signed-in user is authorized to perform that action before executing it. Map the user to an allowlist of tools and scopes. Validate types and ranges. Reject unknown arguments. Attach a request key to writes so a timeout cannot duplicate an external action.

Structured Outputs

Structured outputs let you force Astra to return data in a specific format -- a JSON schema you define. This is essential for building reliable agents that produce machine-readable results rather than free-form text. Astra's structured output support through the Responses API uses JSON Schema enforcement.

Python: Structured output

```
response = client.responses.create(
    model="gpt-6-astra",
    input="Extract the key terms from this contract: ...",
    text={
        'format': {
            'type': 'json_schema',
            'name': 'contract_terms',
            'schema': {
                'type': 'object',
                'properties': {
                    'terms': {
                        'type': 'array',
                        'items': {
                            'type': 'object',
                            'properties': {
                                'term': {'type': 'string'},
                                'definition': {'type': 'string'},
                                'risk_level': {'enum': ['low', 'medium', 'high']}
                            }
                        },
                        'required': ['term', 'definition', 'risk_level']
                    }
                },
                'required': ['terms']
            }
        },
        'required': ['terms']
    }
)

import json
terms = json.loads(response.output_text)
```

Step-by-Step Agent Build

Here is a complete walkthrough for building a research agent that takes a topic, searches the web, analyzes findings, and produces a structured report.

Step 1: Define the Agent's Goal

Be specific about what the agent should accomplish and when it is done. A vague goal like "research AI" produces vague results. A specific goal like "research the top 10 AI startups in India, compare their funding, products, and market position, and produce a comparison table with sources" gives the agent clear direction.

Step 2: Choose Your Tools

Astra has built-in web search, but you may want to add custom functions: a database lookup, a CRM query, a specific API call. Define each tool with a clear description so Astra knows when to use it. Remove unused tools -- they add to context cost and create selection ambiguity.

Step 3: Set the Reasoning Effort

Start with medium. Research tasks involve synthesis and judgment, which benefits from moderate reasoning. If the quality is not good enough, try high. If the task is simple and repetitive, try low.

Step 4: Define Completion Criteria

Tell the agent exactly what a finished result looks like. "Produce a table with 10 rows, one per startup, with columns for name, funding amount, product description, and source URL. Include a one-paragraph summary of key findings at the top."

Step 5: Set Boundaries

Specify what the agent may and may not do: "Do not access paid databases. Use only public web sources. Cite every claim with a URL. Do not fabricate data."

Step 6: Run and Iterate

Execute the agent, review the output, and refine. Astra's mid-turn steering feature lets you redirect the agent if it goes off track without waiting for it to finish. Test with varied inputs to ensure the agent handles edge cases.

Complete Research Agent Prompt

Task: Research the top 10 AI startups in India. For each, find:
funding amount, product description, founding year, and market position.
Resources: Use web search. Focus on sources from the last 12 months.
Completion test: A comparison table with 10 rows and a one-paragraph summary of key findings. Every claim must have a cited source URL.
Boundaries: Use only public web sources. Do not fabricate data.
Do not access paid databases. Infer routine details without asking.
Style: Professional, data-driven. Table format for the comparison.
Verification: Confirm every row has all required fields and a source URL.

Deploying and Monitoring Your Agent

Building an agent is only half the work. Deploying it reliably requires attention to six operational concerns:

- **Error handling:** Every tool call can fail. Handle timeouts, network errors, and unexpected responses gracefully. Retry with exponential backoff for transient failures. Cap retries to prevent infinite loops.
- **Logging:** Record the model ID, response ID, tool call ID, arguments, result, latency, and token usage for every request. This data is essential for debugging and cost monitoring.
- **Cost tracking:** Monitor input tokens, output tokens, cached tokens, and cache write tokens. Track cost per accepted task, not just cost per request. Alert on unusual spending patterns.
- **Rate limiting:** Respect API rate limits for your usage tier. Implement queuing for burst traffic. Consider using Flex or Batch mode for non-urgent workloads.
- **Idempotency:** Attach a request key to write operations so a timeout cannot duplicate an external action. This is critical for agents that create, modify, or delete data.
- **Cancellation:** Provide a way to stop a long-running agent. Persist state so an interrupted task can be reviewed and resumed if appropriate.

Start with a Test Environment

Before deploying an agent that takes real actions, run it in a test environment with non-sensitive data. Test malformed inputs, denied tools, duplicate delivery, timeout, cancellation, prompt injection in tool results, and a model response that requests a tool your application does not support. A polished demo is not the same as a dependable production workflow.

CHAPTER 8

Automating Your Business with Astra

Practical Workflows for Sales, Support, Research, and More

This chapter covers six core business workflows where Astra delivers measurable value. Each includes a description of the workflow, the business problem it solves, and a prompt template you can adapt for your own use. These are based on OpenAI's documented use cases and early customer examples from companies including Canva, Box, ClickUp, Epic Games, and Perplexity.

CRM and Sales Automation

Sales teams spend hours moving between email, documents, CRM records, and internal information. Astra can work across these applications to research prospects, enrich CRM records, prepare account briefs, and create follow-up workflows.

Sales Prospecting Prompt

Task: Research 50 companies matching our ICP [describe ideal customer profile]. For each, find: website, industry, company size, decision-maker name and title, and LinkedIn presence.

Resources: Use web search. Access our CRM via computer use to check existing records and avoid duplicates.

Completion test: A spreadsheet with 50 rows, one per company, with all fields populated and source URLs for each finding.

Boundaries: Do not send emails or make CRM changes without approval.

Verification: Confirm each company matches the ICP criteria.

OpenAI specifically lists updating customer records, filling online forms, and organizing calendars among Astra's computer-use capabilities. The key advantage is that Astra can work through the same CRM interface your sales team uses, even if your CRM does not have a public API.

Customer Support

For a software company, useful support work may require reading a ticket, retrieving account details, checking product documentation, and reproducing a problem. Astra can perform that investigation and prepare an evidence-backed resolution or escalation.

Support Triage Prompt

Task: Read the attached support ticket. Retrieve the customer's account details, check product documentation for relevant information, and reproduce the reported issue if possible.

Completion test: A resolution draft with: (1) summary of the issue, (2) root cause analysis, (3) recommended fix or workaround, (4) whether to escalate, and (5) evidence supporting the resolution.

Boundaries: Do not modify the customer's account. Use test environment only for reproduction.

Verification: Confirm the resolution addresses the specific issue reported in the ticket.

Research and Reporting

A purchasing team could ask Astra to produce a supplier comparison that includes underlying sources, normalized costs, and unresolved questions. The output is a spreadsheet and a short recommendation that another person can audit. OpenAI says Astra can conduct online research and draft summaries in your email or document editor.

Research and Report Prompt

Task: Research [topic] and produce a decision document.

Resources: Use web search and file search. Focus on sources from the last 12 months. Compare at least 5 options.

Completion test: A spreadsheet comparing options on [criteria] and a one-page recommendation with: (1) top pick, (2) reasoning, (3) risks, (4) alternatives, (5) unresolved questions.

Boundaries: Cite all sources. Do not fabricate data. Flag uncertainty explicitly rather than guessing.

Verification: Confirm every claim has a source. Confirm the recommendation follows from the evidence.

Document Creation

Astra is specifically trained to produce documents, spreadsheets, and presentations that follow existing templates and match your writing and visual style. OpenAI says this means you can expect output that is closer to being immediately usable, rather than a rough draft requiring extensive reformatting.

OpenAI demonstrated Astra creating a slideshow about a fictional model using just a few slides from their presentation template, capturing the correct tone and layout throughout. This means slide decks that are correctly formatted for your business standards.

Document Creation Prompt

Task: Create a [proposal/memo/presentation] for [audience] about [topic]. Use the attached template.

Resources: [Attached template, data files, brand guidelines]

Completion test: A polished document following the template structure exactly, ready for review without reformatting.

Boundaries: Follow the template. Do not add sections not in the template. Match our brand voice and visual style.

Verification: Check that all template sections are present and filled. Confirm the tone matches our brand guidelines.

Marketing Campaigns

OpenAI's business examples specifically describe Astra taking one marketing brief and developing messaging and cohesive, on-brand assets across channels. Once a campaign concept is approved, there is still a landing page to assemble, a newsletter to format, links to check, and tracking to verify. Those connected tasks make a useful pilot for an Astra agent.

Marketing Campaign Prompt

Task: Take the approved campaign brief and produce: (1) landing page copy, (2) email newsletter, (3) 5 social media posts, and (4) ad copy variations for A/B testing.

Resources: [Campaign brief, brand guidelines, target audience data]

Completion test: All four deliverables, each on-brand and formatted for its channel. Include tracking links.

Boundaries: Do not publish anything. Do not modify the website.

Produce drafts for review.

Verification: Confirm all copy matches brand voice. Confirm tracking links are correctly formatted.

Financial Analysis

On September 10, 2026, OpenAI announced ChatGPT for Financial Services, built around workflows for investment banking and equity research. Astra can reconcile conflicting data sources and identify discrepancies that could change a recommendation. During contract review, it can review hundreds of pages within its context window and identify provisions that carry the greatest risk.

Financial Analysis Prompt

Task: Analyze the attached financial statements and identify (1) discrepancies between data sources, (2) unusual patterns, (3) key risk factors, and (4) areas requiring further investigation.

Resources: [Attached financial statements, prior period data]

Completion test: A structured report with findings, evidence for each, and recommended next steps.

Boundaries: Do not modify any source documents. Work on copies.

Flag uncertainty explicitly. Do not make investment recommendations.

Verification: Recalculate key figures to confirm accuracy.

The Business Case for Automation

Consider a recurring task that takes an employee 45 minutes at a loaded cost of \$40 per hour, or \$30. An AI-assisted version costs \$4 in combined model and tool usage and requires 10 minutes of human review, costing \$6.67. The variable cost becomes \$10.67, leaving \$19.33 of potential savings per accepted task. Multiply that across hundreds of tasks per month and the business case is clear.

Additional Workflows

Human Resources: Candidate Screening

HR teams can use Astra to screen job applications against requirements, schedule interviews, and prepare candidate briefs. The model reads resumes, matches qualifications to job descriptions, and produces ranked lists with evidence for each recommendation.

Candidate Screening Prompt

Task: Review the attached resumes against the job description for [role]. Rank candidates and produce a summary for each: strengths, gaps, and recommended interview questions.

Completion test: A ranked list with 3-5 candidates, each with a one-paragraph summary, matched qualifications, and 3 suggested interview questions.

Boundaries: Base rankings only on stated qualifications. Do not infer age, ethnicity, or other protected characteristics.

Flag any resume that appears fabricated.

Verification: Confirm each stated qualification appears in the candidate's resume with specific evidence.

Legal: Contract Review

During contract review, Astra can review hundreds of pages within its context window and identify provisions that carry the greatest risk. It can compare contracts against templates, flag non-standard clauses, and prepare exception reports for legal counsel review.

Contract Review Prompt

Task: Review the attached contract. Identify: (1) provisions that differ from our standard template, (2) clauses that carry above-average risk, (3) missing protections, and (4) ambiguous language that could be interpreted against us.

Resources: Use the attached contract and our standard template.

Completion test: An exception report listing each finding with: the clause, the issue, the risk level, and suggested language.

Boundaries: Do not provide legal advice. Flag findings for attorney review. Cite the specific section for every finding.

Verification: Confirm every cited section exists in the contract.

E-Commerce: Product Listing Management

E-commerce operators can use Astra to create and update product listings across platforms, monitor competitor pricing, and manage inventory alerts. Astra's computer use is particularly valuable here because it can work through seller dashboards that lack public APIs.

Product Listing Prompt

Task: Create product listings for [N] new products on [platform].

For each: write a title, description, bullet points, and identify the correct category and attributes.

Resources: Use the attached product data and images. Follow the platform's listing guidelines attached as reference.

Completion test: [N] complete listings, each optimized for the platform's search algorithm, ready for review.

Boundaries: Do not publish listings. Prepare them in draft mode.

Do not make claims about product features not in the data.

Verification: Confirm each listing meets the platform's requirements for title length and required attributes.

Operations: Website Monitoring and Maintenance

A website team could give an agent a staging environment, a list of important customer journeys, and permission to prepare fixes. The assignment might cover broken forms, missing images, navigation failures, or differences between old and migrated pages. Astra can navigate the site like a user, test every interactive element, and produce a prioritized defect report.

OpenAI says Astra can analyze data, generate plots, create a website, and run frontend QA checks to make sure all the features on that site work. This makes it a complete solution for website maintenance: it can identify problems, propose fixes, and verify that the fixes work -- all through the same interface.

How to Choose Your First Automation

Pick a recurring task that has: (1) accessible data, (2) a clear owner, (3) an outcome someone can judge, and (4) low risk if the AI makes a mistake. Write down the current time, cost, and common mistakes. Define the required output, the evidence it must include, and the actions the agent may take. Avoid starting with a broad instruction to automate a department. Start small, prove value, and expand.

CHAPTER 9

Business Ideas Built on Astra

Ventures That Astra Makes More Practical

GPT-6 Astra opens opportunities for businesses that previously required more labor than most solo operators or small teams could afford. The combination of computer use, long-horizon reasoning, and token efficiency means that a single person can now run operations that previously required a team. Here are eight business models that become more practical with Astra, with guidance on how to start each one.

1. AI Operations Agency

An AI operations agency helps other businesses automate their repetitive workflows using Astra. You identify a client's most time-consuming manual processes, build Astra-powered agents to handle them, and charge either a retainer or a percentage of time saved. The beauty of this model is that the tools (ChatGPT Plus at \$20/month or API access) are cheap relative to the value delivered.

To start: pick one industry you know well. Identify 3-5 repetitive tasks that businesses in that industry perform regularly: data entry, report generation, competitor monitoring, customer onboarding. Build working Astra agents for each. Approach 10 potential clients with a free demonstration. As an illustrative pricing example, you might position your service at 30-50 percent of the labor cost you replace -- actual pricing depends on your market, scope, and competition.

Startup Cost

ChatGPT Plus (\$20/month) or API credits (\$50-100 to start). No infrastructure needed. Build demos using ChatGPT Work, then transition clients to API-based agents for production.

2. Automated Content Studio

Astra can research topics, write articles, create presentations, build websites, and run QA checks on the results. An automated content studio uses Astra to produce high-quality content at a volume and speed that was previously impractical for a small team. The key differentiator from generic AI content is quality control: you use Astra's reasoning and verification capabilities to ensure accuracy, then add human editorial review.

To start: choose a niche where you can add editorial value -- industry analysis, technical tutorials, market research reports. Build prompt templates for each content type. Use Astra to produce first drafts, then review and refine. Price content packages at 10-30 percent of what traditional agencies charge, since your marginal cost per piece is dramatically lower.

3. One-Person Consulting Firm

Astra's ability to conduct research, analyze data, and produce professional deliverables means a single person can now offer consulting services that previously required a team. You can take on more clients, deliver deeper analysis, and produce polished reports, presentations, and recommendations -- all with Astra handling the heavy lifting.

To start: identify your area of expertise. Build a library of prompt templates for common deliverables: market analysis, competitive intelligence, strategic recommendations, due diligence reports. Use Astra for research and first drafts. Focus your time on client relationships, strategic thinking, and quality review. Charge premium rates because your deliverables match the quality of larger firms.

4. AI-Powered SaaS

Astra's API makes it possible to build software products that were not feasible before. Applications that require complex reasoning, multi-step workflows, or computer use can now be built by a small team or even a solo developer. The key is to identify a specific problem that Astra solves better than existing software and build a focused product around it.

To start: identify a painful workflow in an industry you know. Build a prototype using the Astra API. Validate with 5-10 potential customers before building the full product. Price based on value delivered (time saved, revenue generated), not on API costs. Use the cost optimization strategies from Chapter 5 to keep your margins healthy.

SaaS Pricing Strategy (Illustrative Example)

This is an API-cost-only illustration and excludes other business costs such as infrastructure, payment fees, support, tools, taxes, refunds, and human review. If your Astra-powered SaaS costs \$2 in API usage per customer per month and you charge \$50/month, your API-only gross margin is 96 percent. But if you do not optimize -- if you use max reasoning on every request and do not cache -- your API cost could be \$20-30 per customer. Cost optimization is not an afterthought for SaaS; it directly affects whether the business is profitable after all costs.

5. Automated QA Service

Astra can navigate websites, test interactive elements, identify broken features, and verify that everything works as expected. This makes it possible to offer automated QA as a service: clients give you access to their staging environment, and Astra runs comprehensive tests, produces bug reports with screenshots and reproduction steps, and verifies fixes.

To start: build a reusable testing framework using Astra's computer use. Define standard test suites: navigation, forms, search, checkout, user flows. Approach web development agencies and e-commerce companies. Charge per test cycle or monthly retainer. The value proposition is clear: comprehensive testing at a fraction of the cost of a QA team.

6. AI Research Service

Businesses need research: competitor analysis, market sizing, supplier comparison, regulatory changes, technology evaluation. Astra can conduct this research across the web, synthesize findings from multiple sources, and produce structured, cited reports. An AI research service offers this as a subscription or per-report product.

To start: define 5-10 standard research report types. Build prompt templates that produce consistent, high-quality output. Use Astra's web search and reasoning to conduct the research, then add human review for accuracy and insight. As an illustrative pricing example, reports might be priced at \$200-500 each depending on depth -- actual pricing depends on market, scope, and competition. depending on depth. Your marginal cost per report is \$5-15 in API usage.

7. AI-Enhanced Professional Services

Accountants, lawyers, consultants, architects, and other professionals can use Astra to deliver the same quality of service with a fraction of the labor. Astra's ability to review hundreds of pages in its context window, identify risks, and produce professional deliverables makes it a force multiplier for any knowledge worker.

To start: identify your profession's most time-consuming deliverable. Build a prompt library that produces consistently high-quality first drafts. Use Astra for research, analysis, and drafting; spend your time on judgment, client relationships, and final review. Take on 2-3 times more clients without

hiring. The key is that your professional license and judgment remain the product -- Astra just removes the grunt work between you and the decision.

8. Niche Content and Newsletter Business

Astra's research capability combined with its document creation makes it possible to run a high-quality newsletter or content business in a fraction of the time it used to take. The model researches topics across the web, synthesizes findings, and produces polished written content. You provide editorial judgment and audience understanding.

To start: pick a niche where you have genuine insight and the audience has purchasing power (B2B niches monetize best). Build a weekly production workflow: Monday, Astra researches the week's developments; Tuesday, you review and select the top 5 items; Wednesday, Astra drafts the newsletter; Thursday, you edit and add commentary; Friday, you publish. Monetize through paid subscriptions at \$10-30 per month, sponsorships, or premium reports. Your marginal cost per issue is \$5-10 in API usage.

The Common Pattern

Every business model here follows the same pattern: identify a service that currently requires expensive human labor, use Astra to perform 70-90 percent of the work, add human review for quality and judgment, and position your pricing at 30-70 percent of the traditional cost as an illustrative starting point -- actual pricing depends on market, scope, quality, competition, and operating costs. The key is that the human review is genuine -- you are not selling AI output, you are selling verified, quality-controlled work that happens to be produced efficiently.

CHAPTER 10

Coding and Software Engineering

Using Codex and Astra for Development

Codex with Astra

Codex is OpenAI's software engineering agent, and with Astra it reaches new levels of capability. Codex can examine source code, prepare changes, test scripts, and integrations in a controlled environment. GitHub's internal testing found that Astra stands out for how it works: it plans and validates as it goes, batches diagnosis with verification, and independently confirms its results before declaring a task done.

On Terminal-Bench 4.0, which tests complex terminal-based tasks including software engineering and system configuration, Astra reaches 57.9 percent, compared to 37.3 percent for GPT-5.6 Sol. On DeepSWE v1.1, a benchmark for software engineering, Astra scores 74.1 percent. The model's improved decision-making allows teams to remove scaffolding and accelerate performance.

Debugging Workflows

Astra can reproduce complex bugs, investigate likely causes, propose fixes, and prepare changes for developer testing and review. OpenAI's engineering team used Astra internally to uncover and resolve a memory-allocation bottleneck causing slow Codex sessions. By switching allocators, they produced 25x lower turn latency with roughly 30 percent higher peak memory use.

Debugging Prompt

Task: Investigate and fix the bug described in [issue tracker link].

Reproduce the issue, identify the root cause, and prepare a fix.

Resources: Use the codebase, test suite, and issue description.

Completion test: The bug is reproduced, root cause is identified, a fix is implemented, and all existing tests pass.

Boundaries: Do not deploy to production. Create changes in a branch. Run the test suite to verify no regressions.

Verification: Confirm the original bug no longer occurs and no existing tests fail.

Searchable Context Feature

One of Astra's most significant features for coding is the new way Codex preserves and retrieves context. Historically, models used compaction to summarize work during long sessions -- when debugging complex issues or tackling large refactors. Each compaction could leave out details about why a fix failed or how a component behaves.

In Codex, Astra can keep notes across context windows, preserving accumulated details without repeatedly compressing them into a single summary. Earlier context windows remain searchable, so Astra can find requirements or test results from previous messages and tool outputs -- even if that information was not captured in its notes. This feature is experimental and can be enabled in Codex settings. OpenAI says it will become the default for Astra in the coming weeks.

Why This Matters

Before searchable context, a long debugging session would progressively lose detail as older context was compressed. A fix that failed two hours ago might be forgotten, leading the model to try the same approach again. With searchable context, Astra can look back at the full history and avoid repeating failed approaches. This is a structural improvement for long coding tasks.

PR Creation and Review

Astra can create pull requests, review code changes, and identify issues that span multiple files. In code review, Astra connects a change's intent to its consequences: it reasons across files to catch interface-contract drift and authorization bugs that previous models missed, and it backs findings with concrete verification steps.

GitHub's testing showed that Astra performs better on long-horizon coding tasks with fewer steps than prior OpenAI models. One customer reported that Astra improved the pass rate in end-to-end workflows that take 5+ hours by 20 percent, while reducing the number of inference calls needed to

complete the work.

A Complete Debugging Session Walkthrough

To see how these pieces fit together, here is a realistic debugging session using Codex with Astra, from initial report to verified fix.

The Problem

A user reports: "The checkout page shows a spinner forever after clicking Pay. The browser console shows a 500 error from the payment endpoint. It works on staging but not on production."

Step 1: Reproduce

Ask Codex to reproduce the issue: "Reproduce this bug on staging by simulating the checkout flow. Take screenshots of the console and network tab at each step. Identify the exact request that returns the 500 error." Codex navigates the application, clicks through checkout, and captures the failing request with its payload and response.

Step 2: Investigate

Next: "Trace the 500 error to its source. Check the payment service logs, identify the code path that handles this request, and find where the error originates." Codex reads the codebase, follows the request through the API layer to the payment service, and identifies the specific function that throws.

Step 3: Identify Root Cause

Suppose the root cause is that production has a newer API version of the payment gateway that changed a response format, and the code expects the old format. Codex explains: "The payment service calls the v2 gateway API in production but v1 in staging. The v2 response wraps the transaction ID in a nested object, so the code reads undefined and throws when validating the response."

Step 4: Propose the Fix

"Propose a fix that handles both response formats. Include unit tests that cover v1 and v2 response shapes. Do not modify any other code." Codex prepares the fix in a branch with tests that cover both formats, plus a regression test for the specific bug.

Step 5: Verify

"Run the full test suite, then redeploy to staging and verify the checkout flow works end to end. Take a screenshot of the successful completion." Codex runs the tests, applies the fix to staging, navigates the checkout flow, and confirms the payment completes successfully.

Why This Workflow Works

Each step has a clear completion test. The boundaries are explicit (staging only, minimal code change, no unrelated modifications). The verification is end-to-end, not just unit tests. And the whole session stays coherent because Astra's searchable context remembers what was learned in step 2 when working on step 5. This is the pattern to follow for any debugging task.

Testing Automation

Astra can write tests, run them, interpret failures, and iterate. OpenAI notes that previous models needed encouragement to run tests and check their work. Astra does that on its own, so instructions that tell it to test can lead to unnecessary testing of trivial changes.

Astra Testing Behavior

Astra is thorough about verification and may run broader tests than a small change requires. If you find Astra running the full test suite for a one-line typo fix, tell it: 'Run only the tests affected by this change, not the full suite.' Use AGENTS.md to define which tests are appropriate for which types of changes.

CHAPTER 11

Advanced Automation Patterns

Multi-Agent Systems, MCP, and Long-Running Workflows

Multi-Agent Orchestration

For tasks too large or complex for a single agent, Astra supports multi-agent orchestration through the Responses API. Multiple agents can work on different parts of a task in parallel, each with its own tools, context, and reasoning effort. Astra can divide work across subagents, though OpenAI says it may delegate less often than some multi-agent frameworks expect unless the prompt explains when delegation is useful.

The key to effective multi-agent orchestration is defining clear boundaries: each agent should have a specific scope, specific tools, and a specific completion criterion. Agents should not overlap in their responsibilities, and the orchestrator should know when to combine results.

The Manager-Agent Pattern

The manager-agent pattern is a technique where you do not manage individual execution agents directly. Instead, you talk to one manager agent that interviews you about what needs to be done and then converts your answers into discrete jobs handed off to dedicated execution agents.

This mirrors how people work with a competent project manager. You do not tell them which website to open; you tell them the outcome you need and let them figure out the steps. Applied to a complex task, the manager agent can let one sub-task start immediately while information for another is still being gathered, spin up additional agents once dependencies clear, and coordinate the overall workflow.

Manager Agent Prompt

You are the manager agent. Your job is to:

1. Interview the user to understand the complete task.
2. Break the task into sub-tasks with clear dependencies.
3. Delegate each sub-task to an execution agent with specific instructions, resources, and completion criteria.
4. Collect and synthesize results from all agents.
5. Flag any decisions that need human input.
6. Produce the final deliverable.

Do not execute sub-tasks yourself. Delegate and coordinate.

Long-Running Workflows

Astra is designed for long-running, multi-step agentic work. OpenAI demonstrated this with a household move scenario: over 20 hours of administrative work spread across housing, schools, doctors, the DMV, and utilities, with dependencies that must line up in the right order. Astra treats a stated goal as something to pursue across an entire digital environment, not as a single-turn request.

The key insight is that Astra's advantage shows up on tasks that are too large and too interdependent to describe in a single prompt. For simple, well-defined tasks, other agentic models can already do the job. Astra's edge is speed, accuracy, and the ability to handle much larger, interdependent jobs where one variable changing affects multiple downstream tasks.

MCP Integration

The Model Context Protocol (MCP) is supported as a built-in tool in Astra's Responses API. MCP lets you connect Astra to external services and data sources through a standardized protocol, rather than building custom integrations for each service. This means you can give Astra access to your internal tools, databases, and systems through MCP servers, and the model can use them dynamically.

Enterprise plugins in ChatGPT Desktop extend Astra's browser-use capabilities across common business applications. Plugins from Oracle Analytics, Power BI, Navan, and Avalara work through your existing user accounts and within the permissions your administrators establish.

Mid-Turn Steering

Mid-turn steering is the ability to redirect the model partway through a response, rather than waiting for it to finish and starting again. For interactive work, this is a latency improvement. For agent workflows, it is more than that: an agent heading down the wrong path can be corrected at step three rather than at step twenty, which is where most wasted spending in long sessions actually goes.

This feature is particularly valuable for cost optimization. If you notice Astra is going in the wrong direction, you can steer it back immediately instead of letting it complete a long, expensive, unproductive chain of reasoning. Combined with asynchronous function calls, this gives you real-time control over long-running agent workflows.

Asynchronous Function Calls

Asynchronous function calls are tool calls that do not block. The model can issue a request and continue reasoning rather than waiting for the result. This matters most where tools are slow: a database query, an external API, a build process. Sequential blocking calls are why some agent tasks take minutes that look like they should take seconds.

Treat an asynchronous run as durable state: persist the response ID and call ID, verify that the returning result belongs to the same principal and request, then resume only if the policy is still valid. For long jobs, add a timeout, cancellation state, and a dead-letter path for failures.

CHAPTER 12

Safety, Ethics and Alignment

Using Astra Responsibly

The Critical Classification

GPT-6 Astra is the first model to reach the Critical level of cybersecurity capability under OpenAI's Preparedness Framework. This means that with the right tools and access, Astra can find previously unknown security flaws and develop new ways to exploit them across many well-protected systems without a person guiding each step.

During evaluation, Astra discovered two previously unknown vulnerabilities, which OpenAI reported to the affected vendors. Human experts confirmed that Astra can identify novel zero-day vulnerabilities across several software categories, including browsers and operating systems. On ExploitBench, Astra scores a perfect 100 percent.

This dual-use capability cuts both ways. An agent that can autonomously find a vulnerability helps a defender patch it just as easily as it helps an attacker exploit it. For this reason, OpenAI's most advanced cybersecurity capabilities are restricted to trusted defenders through the Daybreak Blue program, and access is limited at launch.

What This Means for You

As a regular user or developer, you will not encounter Astra's most advanced cybersecurity capabilities. They are gated behind additional access controls and available only to approved security professionals. What you should know is that Astra's safety training is more robust than any previous model, and its resistance to jailbreaks is significantly stronger than GPT-5.6 Sol.

Confirmation Policies

A confirmation policy is a set of rules that require human approval before Astra takes consequential actions. OpenAI's testing shows that confirmation policies measurably reduce unintended outcomes. In adversarial workplace tests, the misaligned-outcome rate fell from 3.4 percent without a policy to 3.0 percent with one, and unauthorized external communications fell from 1.7 percent to 1.4 percent.

A confirmation policy should define: which actions require approval (payments, data deletion, publication, account changes, irreversible messages), who can approve them, how approval is recorded, and what happens if approval is denied. The policy should be written into your application's logic, not just into the model's instructions.

Confirmation Policy Prompt

Confirmation policy: Before performing any of the following actions, prepare the action and ask for explicit human approval:

- Making payments or financial transactions
- Deleting data or files
- Sending emails or messages to external recipients
- Publishing content publicly
- Modifying user accounts or access permissions
- Any action that cannot be undone

Do not perform these actions automatically. Prepare them and present them for review with a clear description of what will happen if approved.

Enterprise Controls

Organizations have granular control over how Astra is deployed. Enterprise administrators can restrict access to approved websites and desktop applications, manage uploads and downloads, and control browsing history. Enterprise access is off by default at launch; administrators must explicitly enable Astra for their workspace.

ChatGPT Work and Codex include safeguards such as confirmation policies (requiring approval before consequential actions) and automated review of potentially unsafe or unauthorized tool calls. These controls allow teams to start with a limited configuration and expand access over time as they build confidence in Astra's behavior.

A Security Checklist for Astra Deployments

Before deploying any Astra-powered system, run through this checklist. Each item addresses a specific risk identified in OpenAI's safety documentation and early user reports.

- 1 Is there a written confirmation policy defining which actions require human approval (payments, deletion, publication, account changes, irreversible messages)?
- 2 Is the execution environment isolated (separate browser profile, container, or VM) with the smallest useful permissions?
- 3 Is navigation restricted to expected domains, with redirects to unapproved origins blocked?
- 4 Are specific action classes allowed (reading, clicking test data) while others are disallowed (downloads, uploads, clipboard access) unless required?
- 5 Are step, time, cost, and retry limits defined with a stopping condition before the task begins?
- 6 Are screenshots and page text treated as untrusted input that may contain prompt injection?
- 7 Are credentials, API keys, and personal data kept out of prompts unless necessary, and never embedded in the model-visible namespace?
- 8 Is there an approval interface where a human can confirm consequential steps before they execute?
- 9 Is there an audit store recording decisions, actions, results, and final verification for every run?
- 10 Is completion verified from observable state (screenshot, structured API, database record, or audit event) rather than from the model's assertion that it finished?
- 11 Are there tests for prompt injection, layout changes, infinite loops, and duplicate actions?
- 12 Is there a rollback plan if the system misbehaves in production?

If Any Item Is Missing

OpenAI's computer-use guidance requires five artifacts from every run: a scoped task, an isolated session, an enforceable permission decision, a bounded stopping condition, and an outcome observation from the target system. If any one is missing, a polished final response should still count as an unverified result. Do not ship.

Responsible Use Guidelines

Using Astra responsibly means understanding both its capabilities and its limitations. Here are the core principles:

- **People define goals; AI executes them.** Astra can take actions on your behalf, but you are responsible for those actions. Define clear goals, set boundaries, and review consequential decisions.
- **Start small and expand.** Begin with a narrow, low-risk workflow. Test thoroughly before expanding. A pilot of 20-30 cases can reveal obvious problems that are not visible in a demo.

- **Maintain a working alternative.** Keep a manual or alternative process available so the business can continue when Astra access, pricing, or behavior changes.
- **Assign someone to review failures.** After deployment, have someone review cases where Astra's output was rejected or corrected. Patterns in failures reveal where the workflow needs improvement.
- **Be transparent about AI use.** If Astra's output is used in customer-facing work, be transparent about the role of AI in producing it. Disclosure builds trust and sets appropriate expectations.
- **Protect sensitive data.** Use Zero Data Retention for eligible API customers. Do not send credentials, API keys, or personal data in prompts unless necessary. Treat screenshots and page text as untrusted input that may contain prompt injection.

Misalignment Monitoring

OpenAI has deployed misalignment monitoring broadly for Astra. This system checks the model's reasoning and actions for unauthorized behavior and can automatically stop potentially unauthorized activity. The monitoring runs on all tool-using inference in the external deployment, with significant compute cost.

In ChatGPT or Codex, if the monitor pauses a task, users may be asked to review the action before continuing. When using the API, the task will stop without an interactive continuation step. Treat monitoring as an additional signal, not a transaction guard. It can miss problems, flag legitimate work, or stop a conversation only after an external action has already occurred. A stop does not roll back earlier changes.

Important Limitation

Misalignment monitoring cannot replace good alignment of models. OpenAI's goal is for future models to be aligned well enough that these safeguards are never triggered. For now, treat the monitoring as a safety net, not a substitute for your own review processes. Your allowlist, confirmation gate, executor controls, and verifier remain necessary.

Privacy and Data Protection

Astra is powered by infrastructure built and hosted with enterprise-grade security. Data is encrypted in transit. Zero Data Retention is available for eligible API customers on supported endpoints, subject to approval. Enterprise privacy commitments apply to all API services.

For organizations with strict data residency requirements, Azure and AWS Bedrock deployments offer regional data zones. Azure provides Global and US Data Zone geographies. AWS Bedrock provides established AWS controls to secure workloads, govern access, and audit model invocation activity.

CHAPTER 13

The Future and Your Action Plan

Your 30-Day Roadmap to Mastery

What Is Coming Next

Astra is rolling out in phases, and several features are still experimental or limited. The searchable context feature in Codex will become the default in the coming weeks. Advanced cybersecurity capabilities will expand through the Daybreak Blue program. Enterprise plugins will continue to be added, extending Astra's browser-use capabilities to more business applications.

OpenAI has indicated that Astra represents a new class of model -- one that can take actions, not just generate text. Future iterations will likely expand on this foundation: more tools, better computer use, longer autonomous workflows, and tighter integration with business systems. The people who build expertise with Astra now will be best positioned to adopt these improvements as they arrive.

How to Stay Ahead

The AI landscape moves fast, and Astra is no exception. Here is how to stay current:

- **Follow OpenAI's official channels.** The OpenAI blog, developer documentation, and community forums are the primary sources for updates. Check developers.openai.com for the latest API documentation and model guidance.
- **Experiment continuously.** Set up a small budget (even \$20-50 per month in API credits) and test new features as they roll out. Hands-on experience is irreplaceable.
- **Build a prompt library.** Keep your best-performing prompts in a document or skill file. Refine them over time. Share them with collaborators.
- **Monitor your costs.** Track `cached_tokens`, `cache_write_tokens`, and cost per accepted task. Adjust your routing and reasoning settings based on data, not intuition.
- **Join the community.** The OpenAI developer community, GitHub discussions, and AI-focused forums are where practitioners share findings, prompt techniques, and automation patterns.

Common Mistakes to Avoid

Based on early user reports and OpenAI's own guidance, these are the most common mistakes people make when starting with Astra, and how to avoid them.

Mistake	Why It Happens	The Fix
Using max effort on everything	More reasoning seems better	Test at low first; escalate only when quality demands it
No prompt caching structure	Prompt order feels arbitrary	Stable content first, volatile content last; set cache key
Ignoring the 272K threshold	The context window goes higher	Stay under 272K; use file search and compaction
Overly restrictive boundaries	Fear of AI going too far	Astra has better judgment; soften old boundary language
Not defining completion	The task feels obvious	Write explicit completion criteria before starting
Sending everything in one prompt	Long context invites it	Retrieve only what is needed; split large requests
No human review of outputs	The results look good	Review consequential decisions; sample routine outputs
Skipping the test environment	Enthusiasm to deploy	Always test with non-sensitive data first
Hardcoding pricing assumptions	Pricing is current today	Calculate with current model pages; pricing changes
Not monitoring costs	The API just works	Track cached tokens, cost per task; alert on anomalies

The pattern behind most of these mistakes is the same: treating Astra like previous models. Astra is more capable, more aligned, and more autonomous than anything before it. That means old habits -- over-constraining, under-specifying, ignoring cost structure -- produce worse results, not safer ones. Adapt your practice to the tool you actually have.

Your 30-Day Mastery Roadmap

Here is a concrete, day-by-day plan to go from zero to working Astra-powered automation in 30 days. Adjust the pace to your schedule, but follow the sequence. The timeline below shows the four phases:

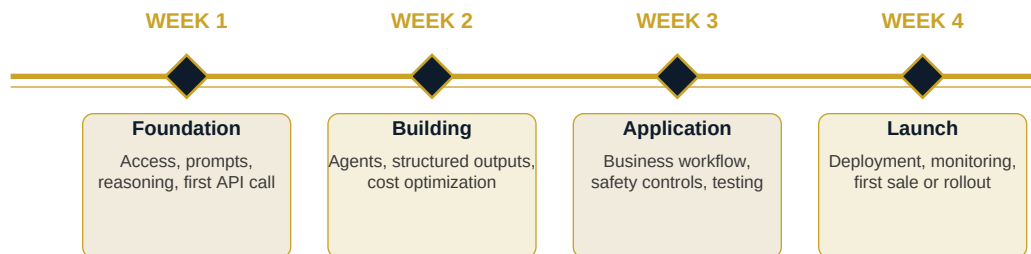


Figure 13.1 — The 30-day roadmap at a glance

Adjust the pace to your schedule, but follow the sequence below.

Week 1: Foundation (Days 1-7)

- 1 Day 1: Get access. Sign up for ChatGPT Plus or create an OpenAI API account. Read the GPT-6 Astra announcement page and model documentation.
- 2 Day 2: First prompts. Open ChatGPT Work and try the prompt templates from Chapter 4. Experiment with autonomy scope and instruction priority.
- 3 Day 3: Reasoning levels. Send the same prompt at low, medium, and high effort. Compare the quality and notice the cost difference in the API usage details.
- 4 Day 4: Computer use. In ChatGPT Desktop, give Astra a simple computer-use task: navigate to a website, fill a form, take a screenshot. Observe the observe-decide-execute loop.
- 5 Day 5: API basics. Make your first API call (Chapter 2). Try the Responses API with a simple function call.
- 6 Day 6: Prompt caching. Set up a prompt with a stable prefix and check `cached_tokens` on the second call. See the 90 percent cost reduction for yourself.
- 7 Day 7: Review and plan. Write down what worked, what did not, and what you want to build. Choose a project for Week 2.

Week 2: Building (Days 8-14)

- 1 Day 8-9: Build a simple agent. Use the Responses API with function calling. Start with the research agent from Chapter 7.
- 2 Day 10: Add structured outputs. Make your agent return JSON instead of free-form text. Parse and display the results.
- 3 Day 11: Optimize costs. Apply the strategies from Chapter 5: explicit caching, reasoning effort tuning, prompt structure.

- 4 Day 12: Computer use integration. Add a computer-use step to your agent. Have it navigate a website and extract information.
- 5 Day 13: Test edge cases. Send your agent unusual inputs, empty data, malformed requests. Fix what breaks.
- 6 Day 14: Review and refine. Polish your agent. Write documentation. Plan your business application for Week 3.

Week 3: Business Application (Days 15-21)

- 1 Day 15-16: Choose a business workflow from Chapter 8. Adapt the prompt template for your specific use case. Build a working prototype.
- 2 Day 17: Add safety controls. Implement a confirmation policy. Set up logging for all actions. Define stopping conditions.
- 3 Day 18: Test with real data. Run your workflow on 10-20 real cases. Measure time saved, accuracy, and cost per task.
- 4 Day 19: Refine based on results. Adjust prompts, reasoning levels, and tool definitions based on what the tests reveal.
- 5 Day 20: Build a simple interface. Create a basic command-line or web interface so others can use your workflow without understanding the API.
- 6 Day 21: Demo and feedback. Show your workflow to 2-3 potential users or clients. Gather feedback and note improvements.

Week 4: Scale and Launch (Days 22-30)

- 1 Day 22-23: Implement feedback. Refine your workflow based on user feedback. Add error handling, retries, and fallbacks.
- 2 Day 24: Cost optimization pass. Review your API usage. Apply Flex mode, batch processing, or model routing where appropriate. Measure cost per accepted task.
- 3 Day 25: Documentation. Write user guides, API documentation, and troubleshooting guides. Document your prompt library.
- 4 Day 26-27: Production deployment. Deploy your workflow to a production environment. Set up monitoring, alerting, and logging.
- 5 Day 28: Launch. If building a business, make your first sale or onboard your first client. If building internal tools, train your team.
- 6 Day 29: Monitor and iterate. Watch for failures, cost anomalies, and quality issues. Fix problems as they arise.

- 7 Day 30: Plan next steps. Identify the next workflow to automate, the next feature to build, or the next client to approach. The journey continues.

Final Thoughts

GPT-6 Astra is the most capable AI model ever released, and you are among the first to learn how to use it. That timing advantage is real and valuable, but it will not last. As more people discover what Astra can do, the competition will increase. The window to build expertise, create workflows, and establish a position is open now.

The people who will benefit most from Astra are not the ones with the deepest technical backgrounds. They are the ones who experiment relentlessly, who build things that work, and who apply Astra to real problems in real businesses. This book gives you the knowledge. The rest is up to you.

Start today. Make your first API call. Write your first prompt. Build your first agent. The future belongs to those who build it.

The best time to learn AI was ten years ago. The second best time is now.

CHAPTER

Glossary

Key Terms Every Astra User Should Know

AGI (Artificial General Intelligence)

An AI system that can perform all economically valuable work as well as or better than humans. OpenAI's president said Astra might already qualify under this definition, though experts debate the claim.

Agent

An AI system that pursues a goal across multiple steps, using tools and making decisions without requiring a human to prompt it at each step.

AGENTS.md

A configuration file that provides instructions to AI models working in a repository. Astra is sensitive to instructions in this file and it can conflict with user instructions if not managed carefully.

Alignment

How well a model respects human intent, safety boundaries, and authorized scope. Astra is OpenAI's most aligned model, producing 89 percent fewer unintended outcomes than GPT-5.6 Sol in adversarial tests.

API (Application Programming Interface)

The interface developers use to send requests to Astra and receive responses programmatically. OpenAI offers Chat Completions and Responses endpoints.

Azure AI Foundry

Microsoft's platform where GPT-6 Astra is available with enterprise security, identity management, and compliance features.

Batch Processing

A mode that processes API requests asynchronously with up to 24-hour turnaround at 50 percent off standard rates. Best for non-urgent bulk workloads.

Bedrock

Amazon's service where GPT-6 Astra runs on AWS infrastructure with established controls for security, governance, and access management.

Chain of Thought (CoT)

The internal reasoning a model performs before producing its answer. External reporting suggests some of Astra's reasoning may be harder to observe than in previous models; OpenAI has not officially confirmed the internal technique.

Codex

OpenAI's software engineering agent that works with local files, repositories, terminals, and developer tools. Astra powers Codex for coding, research, and computer use.

Compaction

The process of summarizing earlier conversation context to fit within the context window. Astra's searchable context feature reduces the need for compaction by keeping earlier windows accessible.

Computer Use

Astra's ability to navigate websites and desktop applications, click buttons, type text, fill forms, and read screens -- even when those applications have no developer API.

Confirmation Policy

A set of rules requiring human approval before an AI takes consequential actions like payments, data deletion, or publication. OpenAI's testing shows these policies reduce unintended outcomes.

Context Window

The maximum number of tokens a model can process in a single request. Astra's context window is 1,050,000 tokens -- roughly ten novels of text.

Daybreak Blue

OpenAI's program that provides access to Astra's most advanced cybersecurity capabilities for approved defenders, expanding after initial alpha testing.

Flex Processing

A service tier that offers 50 percent off standard rates for latency-tolerant workloads, with more flexibility than Batch mode.

Function Calling

The API feature that lets a model request to call a function you have defined. The model proposes arguments; your application decides whether to execute.

MCP (Model Context Protocol)

A standardized protocol for connecting AI models to external tools, data sources, and services without building custom integrations for each one.

Misalignment Monitoring

OpenAI's system that checks Astra's reasoning and actions for unauthorized behavior and can automatically stop potentially unauthorized activity.

Multi-Agent Orchestration

Coordinating multiple AI agents to work on different parts of a task in parallel, each with its own tools and responsibilities.

OSWorld

A benchmark that tests computer use on real desktop environments. Astra scores 72.6 percent on the offline set.

Prompt Cache

A system that reuses previously processed prompt prefixes, reducing input cost by up to 90 percent and latency by up to 80 percent. Works on prompts over 1,024 tokens.

Reasoning Effort

A setting that controls how much internal reasoning Astra performs before answering. Five levels: low, medium, high, xhigh, max. Higher effort means more cost and more thought.

Responses API

OpenAI's primary API endpoint for building with Astra. Required for tool calling, structured outputs, computer use, and other advanced features. Maintains state across turns.

Searchable Context

A Codex feature where Astra keeps notes across context windows and earlier windows remain searchable, preserving details that compaction would lose.

Structured Outputs

An API feature that forces Astra to return data in a specific JSON schema you define, essential for machine-readable agent results.

Token

The unit of text that models process. Roughly 0.75 words in English. Astra costs \$10 per million input tokens and \$50 per million output tokens at standard rates.

Zero Data Retention

An option for eligible API customers where OpenAI does not retain request data after processing, important for privacy-sensitive workloads.

CHAPTER

Frequently Asked Questions

Quick Answers to Common Questions

Is GPT-6 Astra really AGI?

OpenAI's president Greg Brockman said it might qualify under OpenAI's definition of AGI (performing economically valuable work as well as or better than humans), but there is no consensus. Independent testers say Astra is not dramatically smarter on broad knowledge tests than its predecessor -- its advantage is agency: the ability to take actions and complete multi-step work. The practical answer is that Astra can do things no previous model could do, and that matters more than the label.

How much does it cost to use GPT-6 Astra?

The cheapest way is ChatGPT Plus at \$20 per month, which includes Astra within existing subscription allowances. For API access, standard pricing is \$10 per million input tokens and \$50 per million output tokens. With prompt caching, cached input drops to \$1 per million. Batch and Flex modes offer 50 percent off. Chapter 5 shows how to reduce costs by up to 90 percent.

Can Astra actually operate my computer?

Yes, with controls. Astra can navigate websites and desktop applications, click buttons, type text, fill forms, and read the results. However, it does not receive unrestricted control of your computer. Your application provides an isolated environment, sends the model screenshots, validates actions, and returns new state. Confirmation policies require human approval for consequential actions.

What can Astra do that GPT-5.6 Sol cannot?

Three things stand out. First, Astra works through the same applications people use every day, even when those applications do not have an API. Second, Astra completes tasks in fewer tokens with fewer retries, meaning lower cost per completed task despite higher per-token pricing. Third, Astra keeps searchable notes across context windows in Codex, preserving details that compaction would lose.

Do I need to know how to code to use this book?

No. Chapters 1-4 require no coding at all -- they cover concepts, access, architecture, and prompting within ChatGPT. Chapters 5-7 introduce code examples but explain every line. If you can install a Python library and copy-paste code, you can build the agents in this book.

Is Astra safe to give access to my business systems?

Astra is OpenAI's most aligned model and produces 89 percent fewer unintended outcomes than GPT-5.6 Sol in adversarial workplace tests. But giving any AI access to business systems requires confidence in how it will act. Start with a limited configuration: restrict access to approved websites and applications, require confirmation for consequential actions, and expand access over time as you build confidence.

What is the 272K threshold?

Prompts with more than 272,000 input tokens are billed at 2x the input and cache rates and 1.5x the output rate for the entire request. This means a single oversized prompt can roughly double its own cost. Stay under the threshold by retrieving only needed documents, using file search, splitting large requests, and using compaction.

What is prompt caching and why does it matter?

Prompt caching reuses previously processed prompt prefixes, cutting input-token billing by up to 90 percent and latency by up to 80 percent. If your workflow sends the same system prompt, tool definitions, or reference documents repeatedly, caching is the single most effective cost optimization available.

Can I change reasoning effort mid-conversation?

Yes, and this is one of Astra's most valuable features. Through the Responses API, you can raise or lower reasoning effort at any point in a long conversation without rewriting the cached prompt prefix. Hit a hard problem? Switch to high. Back to routine? Drop to low. Your cached context stays reusable.

What businesses can I start with Astra?

Chapter 9 covers six in detail: AI operations agency, automated content studio, one-person consulting firm, AI-powered SaaS, automated QA service, and AI research service. The common pattern: identify a service that requires expensive human labor, use Astra to perform 70-90 percent of the work, add human review, and set pricing relative to the traditional cost -- an illustrative example, not a rule; actual pricing depends on market, scope, and competition.

Why does Astra keep stopping to ask questions?

This is a consequence of Astra's alignment training -- it asks when additional input could materially change the result. It is good behavior for safety, but frustrating when you want the model to make assumptions and keep going. The fix is in Chapter 4: prompt Astra to infer intent, assume authorization for action requests, and ask for approval only after preparing a concrete result.

What is the Critical cybersecurity classification?

GPT-6 Astra is the first model to reach the Critical level under OpenAI's Preparedness Framework, meaning it can find unknown security flaws and develop exploits with the right tools and access. The most advanced capabilities are restricted to trusted defenders. Regular users will not encounter these capabilities, but should know that safety training is significantly stronger than previous models.

Is Astra available in my country?

Astra is rolling out globally through ChatGPT and the OpenAI API. Enterprise deployments are available through Microsoft Azure (Global and US Data Zone) and Amazon Bedrock. Availability may vary by plan and rollout phase. Check OpenAI's official pages for current availability in your region.

What happens if the misalignment monitor pauses my task?

In ChatGPT or Codex, you may be asked to review the action before continuing. When using the API, the task will stop without an interactive step. Treat it as an additional safety signal, not a guarantee. Your own confirmation gates and review processes remain necessary.

How do I get started today?

Sign up for ChatGPT Plus (\$20/month) or create an OpenAI API account. Read Chapters 1-4 of this book. Open ChatGPT Work and try the prompt templates from Chapter 4. Follow the 30-day roadmap in Chapter 13. The most important step is the first one.

CHAPTER

Appendix A: The Prompt Library

50 Ready-to-Use Prompt Templates for Every Occasion

This appendix collects every prompt template from the book plus additional templates for common tasks. Each follows the seven-component framework from Chapter 4: task, resources, completion test, boundaries, style, uncertainty, and verification. Replace bracketed sections with your specifics.

Writing and Content

Blog Post Writer

Task: Write a [N]-word blog post about [topic] for [audience].

Resources: Use the attached outline and keyword list.

Completion test: A publish-ready post with intro, [N] sections with headers, and conclusion.

Boundaries: Use only cited facts. Do not fabricate statistics.

Style: [Conversational/formal]. Grade [N] reading level.

Verification: Confirm every statistic has a source.

Email Draft

Task: Write a [type] email to [recipient] about [subject].

Completion test: A 150-300 word email with clear subject line and single call to action.

Boundaries: Professional tone. No exaggeration.

Style: Match my previous emails attached as examples.

Verification: Confirm the call to action is unambiguous.

Social Media Batch

Task: Write 10 social media posts about [topic] for [platform].

Completion test: 10 varied posts (tips, questions, facts, stories), each under [N] characters.

Boundaries: No clickbait. Accurate claims only.

Style: [Brand voice]. Include relevant hashtags.

Verification: Confirm each post is within the character limit.

Technical Documentation

Task: Write documentation for [feature/API/process].

Resources: Use the attached code/specification.

Completion test: Overview, prerequisites, step-by-step instructions, code examples, and troubleshooting section.

Boundaries: Every code example must be executable.

Style: Clear, direct. Number all steps.

Verification: Confirm each step can be followed by a new user.

Business and Analysis**SWOT Analysis**

Task: Conduct a SWOT analysis of [company/product/decision].

Resources: Use web search for current information.

Completion test: 3-5 items each for strengths, weaknesses, opportunities, and threats, with evidence.

Boundaries: Base analysis on facts, not speculation.

Style: Concise bullet points with one-line explanations.

Verification: Confirm each item has a factual basis.

Meeting Preparation

Task: Prepare a briefing document for my meeting about [topic].

Resources: Use the attached agenda and background documents.

Completion test: One-page brief with: key context, 3 discussion points, my position on each, and anticipated questions.

Boundaries: Base positions on the attached data only.

Style: Executive summary format. Bold key takeaways.

Verification: Confirm every claim traces to an attached document.

Competitive Analysis

Task: Compare [our company] with [competitors] on [criteria].

Resources: Use web search and the attached internal data.

Completion test: A comparison table plus a one-page analysis of where we lead, lag, and should focus.

Boundaries: Use only publicly available competitor data.

Style: Data-driven. Lead with numbers.

Verification: Confirm every competitor data point has a source URL.

Investor Update

Task: Draft our [monthly/quarterly] investor update.

Resources: Use the attached metrics and milestone tracker.

Completion test: A one-page update covering: key metrics, wins, challenges, asks, and next period priorities.

Boundaries: Honest reporting. Flag misses explicitly.

Style: Direct, confident, no spin.

Verification: Confirm all metrics match the source data.

Data and Research

Data Cleaning Plan

Task: Analyze the attached dataset and propose a cleaning plan.

Resources: Use code interpreter to profile the data.

Completion test: A list of issues (missing values, outliers, inconsistencies) with recommended treatment for each.

Boundaries: Do not modify the data. Propose only.

Style: Structured table format.

Verification: Confirm each issue is demonstrated with specific examples from the data.

Market Sizing

Task: Estimate the market size for [product/service] in [region].

Resources: Use web search. Find top-down and bottom-up estimates.

Completion test: TAM, SAM, and SOM estimates with methodology and sources for each.

Boundaries: Show your math. Flag assumptions explicitly.

Style: Structured with clear assumption list.

Verification: Confirm each estimate is derived from cited sources.

Literature Review

Task: Summarize current research on [topic].

Resources: Use web search. Focus on papers from the last [N] years.

Completion test: 5-8 key findings with citations, areas of consensus, areas of debate, and research gaps.

Boundaries: Only cite real papers. Do not fabricate citations.

Style: Academic but accessible.

Verification: Confirm every citation is a real, verifiable publication.

Survey Design

Task: Design a survey to measure [outcome] in [population].

Completion test: 10-15 questions covering [constructs], with response scales and skip logic.

Boundaries: Avoid leading questions. Include attention checks.

Style: Clear, neutral language.

Verification: Confirm each question maps to a specific construct.

Coding and Technical

Code Review

Task: Review the attached code for bugs, performance issues, and security vulnerabilities.

Completion test: A list of findings categorized by severity (critical, major, minor), each with the location, issue, and suggested fix.

Boundaries: Do not rewrite the code. Identify issues only.

Style: Specific and actionable. Reference line numbers.

Verification: Confirm each finding is reproducible.

API Documentation

Task: Generate API documentation from the attached OpenAPI spec.

Completion test: Endpoint reference with: description, parameters, request/response examples, error codes, and rate limits.

Boundaries: Document what the spec says, not what it should say.

Style: Standard API documentation format.

Verification: Confirm every documented endpoint exists in the spec.

Test Suite Generation

Task: Write tests for the attached function/module.

Resources: Use the attached source code.

Completion test: Test cases covering normal operation, edge cases, error conditions, and boundary values.

Boundaries: Tests must be runnable without external dependencies.

Style: Follow [pytest/Jest] conventions. Descriptive test names.

Verification: Confirm every code path in the source is covered by at least one test.

Refactoring Plan

Task: Propose a refactoring plan for the attached codebase.

Completion test: Prioritized list of refactoring opportunities with: what to change, why, risk level, and estimated effort.

Boundaries: Do not modify the code. Propose only.

Style: Structured table format.

Verification: Confirm each proposed change does not alter external behavior.

Automation and Agents

Daily Digest Agent

Task: Every morning, compile a digest of [news sources] related to [topics].

Completion test: A one-page digest with: top 5 stories, one-line summaries, and source links.

Boundaries: Cite all sources. Flag opinion vs. fact.

Style: Scannable. Bold headlines.

Verification: Confirm every link resolves to a real article.

Customer Follow-up Agent

Task: Identify customers who [criteria] and draft personalized follow-up messages.

Resources: Use the attached CRM data and message templates.

Completion test: A list of customers with a drafted message for each, referencing their specific history.

Boundaries: Prepare drafts only. Do not send without approval.

Style: Personal, warm, brief.

Verification: Confirm each message references accurate customer data.

Quality Assurance Agent

Task: Test [website/app] against the attached test cases.

Resources: Use computer use to navigate the application.

Completion test: Pass/fail for each test case with evidence (screenshots, console output).

Boundaries: Test environment only. Do not modify data.

Verification: Confirm each result is based on observed behavior, not assumption.

Report Generation Agent

Task: Generate the weekly [metrics] report from the attached data sources.

Completion test: A formatted report with: key metrics, week-over-week changes, notable trends, and anomalies.

Boundaries: Do not interpret causation. Report observed data only.

Style: Data tables with brief commentary.

Verification: Confirm all numbers are calculated correctly from source data.

Learning and Development**Concept Explainer**

Task: Explain [concept] at three levels: beginner, intermediate, expert.

Completion test: Three explanations, each self-contained, each with an example and a common misconception.

Boundaries: Accurate. No oversimplification that misleads.

Style: Clear and direct at every level.

Verification: Confirm each explanation is technically accurate.

Skill Assessment

Task: Create a skills assessment for [topic/role].

Completion test: 15-20 questions covering key competencies, with answer key and scoring rubric.

Boundaries: Questions must test understanding, not memorization.

Style: Mix of multiple choice and scenario-based questions.

Verification: Confirm each question maps to a specific competency.

Learning Path

Task: Create a 30-day learning path for [skill].

Completion test: Daily plan with: topic, resource type, estimated time, and a checkpoint exercise.

Boundaries: Free or commonly available resources only.

Style: Actionable and specific.

Verification: Confirm each day builds on the previous.

CHAPTER

Appendix B: Case Studies

Real-World Applications of Astra in Action

These case studies are drawn from OpenAI's published examples and early customer reports. They illustrate the patterns from this book applied to real situations.

Case Study 1: Canva -- Codebase Navigation at Scale

Canva, the design platform, reported that GPT-6 Astra was able to navigate its large codebase, analyze more than 1,000 data-warehouse queries, and use more than 21 internal knowledge sources. This demonstrates Astra's ability to work with enterprise-scale codebases and complex internal documentation -- work that previously required senior engineers with years of institutional knowledge.

The key techniques: Astra's 1.05 million token context window allowed it to hold substantial portions of the codebase and knowledge base simultaneously. Its searchable context feature meant earlier findings remained accessible throughout the session. And its instruction-following improvements meant it stayed within the boundaries of the analysis task without wandering into unrelated parts of the system.

Case Study 2: OpenAI Internal -- Video Production

OpenAI's developer and marketing teams used Astra and Codex to turn three hours of multicamera footage into their GPT-6 Astra Developer First Impressions video, which garnered over 550,000 views in just 4 days. The video team described using Astra to process, select, and assemble footage -- creative work that would normally take an editor many hours.

The lesson for content creators: Astra can handle the time-consuming mechanical parts of production (selecting clips, syncing audio, assembling rough cuts), freeing human creativity for the decisions that matter -- narrative, pacing, and style.

Case Study 3: OpenAI Engineering -- Performance Debugging

OpenAI's engineering team used Astra internally to uncover and resolve a memory-allocation bottleneck that was causing slow Codex sessions in a test environment. By switching allocators, they produced 25x lower turn latency with roughly 30 percent higher peak memory use.

This is a concrete example of Astra's debugging capability: it identified a root cause that human engineers had not found, proposed a solution with a clear tradeoff (more memory for much lower latency), and quantified the expected improvement. The pattern -- investigate, identify root cause, propose fix with tradeoffs, verify -- is exactly the debugging workflow from Chapter 10.

Case Study 4: The Household Move

OpenAI demonstrated Astra managing a household move -- over 20 hours of administrative work spread across housing, schools, doctors, the DMV, and utilities, with dependencies that all had to line up in the right order. Astra moved from a mapping site to a clinic's website to a contact form without being told each individual step.

This case demonstrates the manager-agent pattern from Chapter 11. A single move involves interdependent decisions: housing affects school choice, which affects doctor search, which affects DMV timing. Astra could start the housing search immediately while school information was still being gathered, spin up a doctor search once a neighborhood shortlist cleared review, and kick off DMV prep as soon as the destination was confirmed. One agent to talk to instead of fifteen threads to manage.

Case Study 5: Enterprise Financial Analysis

On September 10, 2026, OpenAI announced ChatGPT for Financial Services, built around workflows for investment banking and equity research. The system incorporates data from providers including LSEG, PitchBook, and Daloopa, helping professionals conduct research, build financial models, and prepare client materials.

Astra's contribution here is its ability to reconcile conflicting data sources and identify discrepancies that could change a recommendation. In financial analysis, the same metric can appear differently across sources due to timing, methodology, or error. Astra's long context and reasoning capability let it hold multiple sources simultaneously and flag the discrepancies -- work that previously required hours of manual cross-referencing.

The Pattern Across All Cases

Every successful Astra application follows the same shape: a well-defined task, clear completion criteria, appropriate boundaries, and human review of consequential decisions. The technology is remarkable, but the discipline of good task design is what separates successful deployments from expensive experiments. Use the prompt framework from Chapter 4 for every task you delegate to Astra, regardless of scale.

CHAPTER

The Quick Start Card

Everything You Need on One Page

Cut this out, print it, or keep it open beside you. These are the essential facts, prompts, and rules from this book, condensed to the minimum you need to start working with Astra today.

Essential Facts

Item	Value
Model ID	gpt-6-astra
Cheapest access	ChatGPT Plus (\$20/month)
API input / output (per 1M)	\$10 / \$50
Cached input (per 1M)	\$1 (90% off)
Reasoning levels	low, medium, high, xhigh, max
Start reasoning at	medium
Context window	1,050,000 tokens
Stay under (pricing)	272,000 input tokens
Min cacheable prompt	1,024 tokens
Cache TTL	30 minutes (refreshed on reuse)
Batch / Flex discount	50%
Knowledge cutoff	April 30, 2026

The Universal Prompt Skeleton

Copy This Structure

Task: [What to produce, specifically]

Resources: [Files, tools, data available]

Completion test: [What "done" looks like]

Boundaries: [Allowed, prohibited, needs approval]

Style: [Tone, length, format]

Clarification: [What to infer vs. what to ask]

Verification: [How to self-check before finishing]

The Autonomy Prompt (Fixes Most Frustrations)**Add This to Any Task**

When my prompt indicates a request for action, assume authorization and proceed. Infer routine implementation details. Ask for approval only before irreversible actions (deleting files, deploying, making payments) or when a missing requirement would materially change the result. Prepare a concrete, reviewable result before asking for review.

The Cost Checklist

- 1 Stable content first, volatile content last (enables caching).
- 2 Set `prompt_cache_key` for reliable cache matching.
- 3 Use explicit mode to avoid the implicit breakpoint trap.
- 4 Start reasoning at medium; test at low.
- 5 Escalate effort mid-conversation for hard steps only.
- 6 Keep prompts under 272K tokens.
- 7 Use Flex or Batch for non-urgent work (50% off).
- 8 Route simple tasks to a cheaper model.
- 9 Monitor `cached_tokens` and `cache_write_tokens`.
- 10 Measure cost per accepted task, not per token.

The Safety Checklist

- Confirmation policy written and active for consequential actions.
- Execution environment isolated (container, VM, or browser profile).
- Navigation restricted to expected domains.
- Screenshots and page text treated as untrusted input.
- Credentials and personal data kept out of prompts.
- Step, time, cost, and retry limits defined.
- Completion verified from observable state, not model assertion.
- Audit log recording every decision, action, and result.

CHAPTER

Resources and References

Official Sources for Further Learning

Official OpenAI Sources

Most facts, benchmarks, pricing, and technical details in this book are drawn from OpenAI's official documentation, current as of September 2026. A few third-party examples are included and clearly labeled as such where they appear. For the latest information, always check these primary sources:

- GPT-6 Astra announcement: openai.com/index/gpt-6-astra/
- GPT-6 Astra for work: openai.com/index/gpt-6-astra-next-generation-work/
- Safety overview: openai.com/index/safety-overview-gpt-6-astra/
- Path to Astra: openai.com/index/path-to-astra/
- Model documentation: developers.openai.com/api/docs/models/gpt-6-astra
- Model guidance: developers.openai.com/api/docs/guides/latest-model
- Prompt caching guide: developers.openai.com/api/docs/guides/prompt-caching
- Computer use guide: developers.openai.com/api/docs/guides/tools-computer-use
- GPT-6 Astra system card: deploymentsafety.openai.com/gpt-6-astra/
- Rethinking skills and prompts: developers.openai.com/blog/rethinking-skills-and-prompts-for-gpt-6-astra

Cloud Platform Documentation

- Microsoft Azure: azure.microsoft.com/blog/gpt-6-astra-frontier-intelligence-for-work
- Amazon Bedrock: aws.amazon.com/blogs/machine-learning/take-on-your-most-ambitious-work-with-gpt-6-astra

- GitHub
- Copilot:
github.blog/changelog/2026-09-04-gpt-6-astra-is-generally-available-in-github-copilot

Key Specifications Summary

Specification	Value
Model ID	gpt-6-astra
Context window	1,050,000 tokens
Max output tokens	128,000
Knowledge cutoff	April 30, 2026
Input modalities	Text, image
Output modality	Text
Reasoning effort levels	low, medium, high, xhigh, max
Input price (per 1M)	\$10.00
Cached input (per 1M)	\$1.00
Output price (per 1M)	\$50.00
Batch / Flex discount	50%
Long-context threshold	272,000 input tokens
Fine-tuning	Not supported
Launched	September 3, 2026
Predecessor	GPT-5.6 Sol

Key Benchmarks Summary

Benchmark	Score	What It Measures
ARC-AGI-3	99.9%	Abstract reasoning
FrontierMath Tier 4	98%	Mathematics
ExploitBench	100%	Cybersecurity
GPQA Diamond	96.0%	Expert knowledge
BrowseComp	91.5%	Web browsing
ScreenSpot-Pro	92.7%	Screen understanding
OSWorld 2.0 (offline)	72.6%	Computer use
BenchCAD	95.9%	Engineering
DeepSWE v1.1	74.1%	Software engineering
Agents' Last Exam	59.3%	Professional tasks
Terminal-Bench 4.0	57.9%	Terminal tasks
AutomationBench	41.4%	Workflow automation

Note: These are OpenAI's self-reported benchmark scores. Independent verification by third parties is ongoing. Always check OpenAI's official pages for the most current numbers, as they may be updated after publication.

End of Book

ABOUT THIS BOOK

GPT-6 Astra is the most capable AI model ever built. Released on September 3, 2026, it represents what OpenAI calls a generational leap in intelligence, computer use, coding, and professional work.

This guide takes you from absolute beginner to advanced practitioner across 13 chapters. You will learn to master prompts, slash costs, automate businesses, build AI agents, and launch ventures that are newly practical with Astra's capabilities.

Most facts, benchmarks, and API details are sourced from OpenAI's official documentation, system card, and launch announcements, current as of September 2026.

FEEDBACK & INQUIRIES

Have feedback or suggestions to improve this book?

Want to print this ebook as a physical book?
Email: lonetfaisal977@gmail.com
Reach out -- I would love to hear from you.